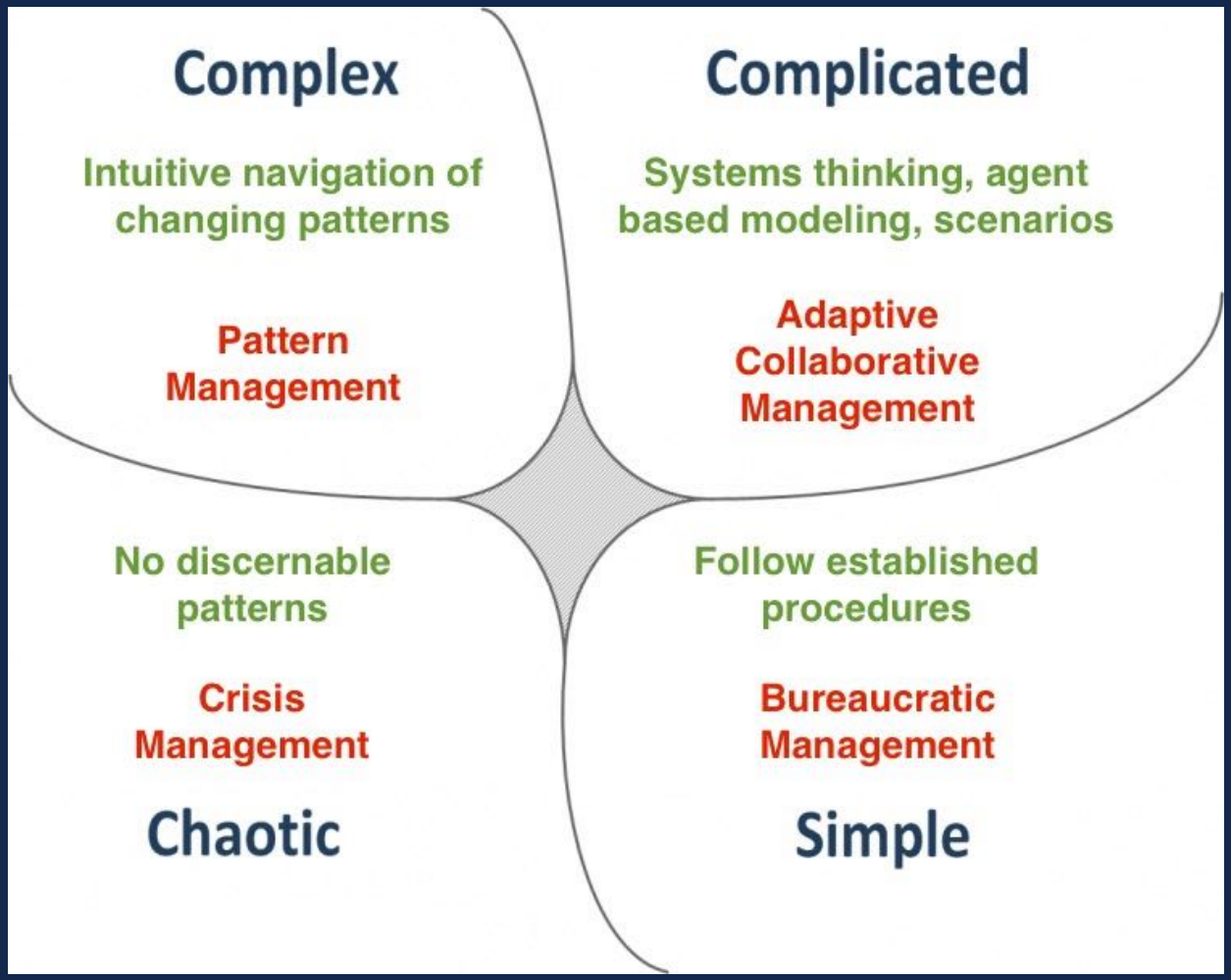
Project Reveal

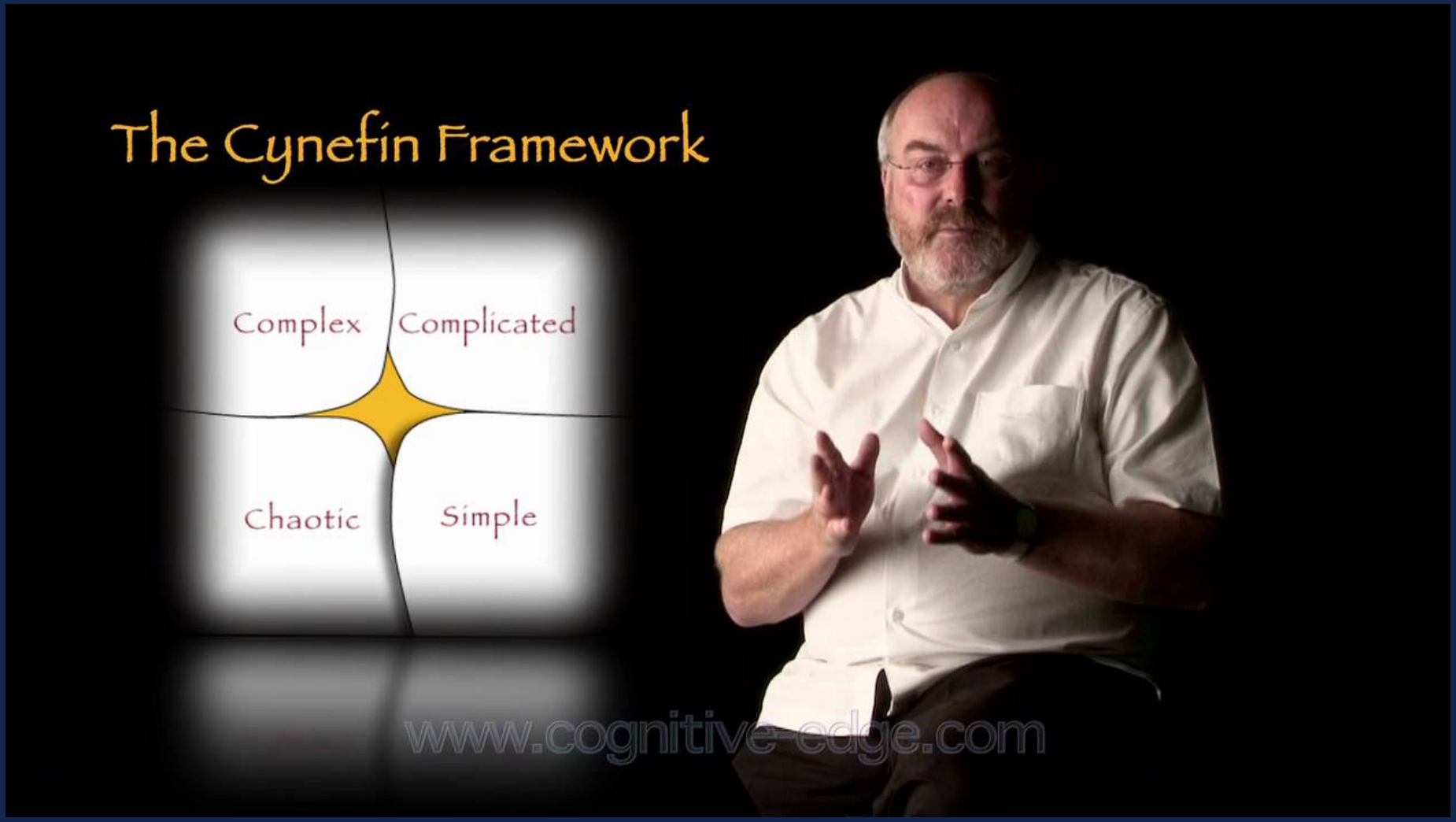
with Thomas Powell

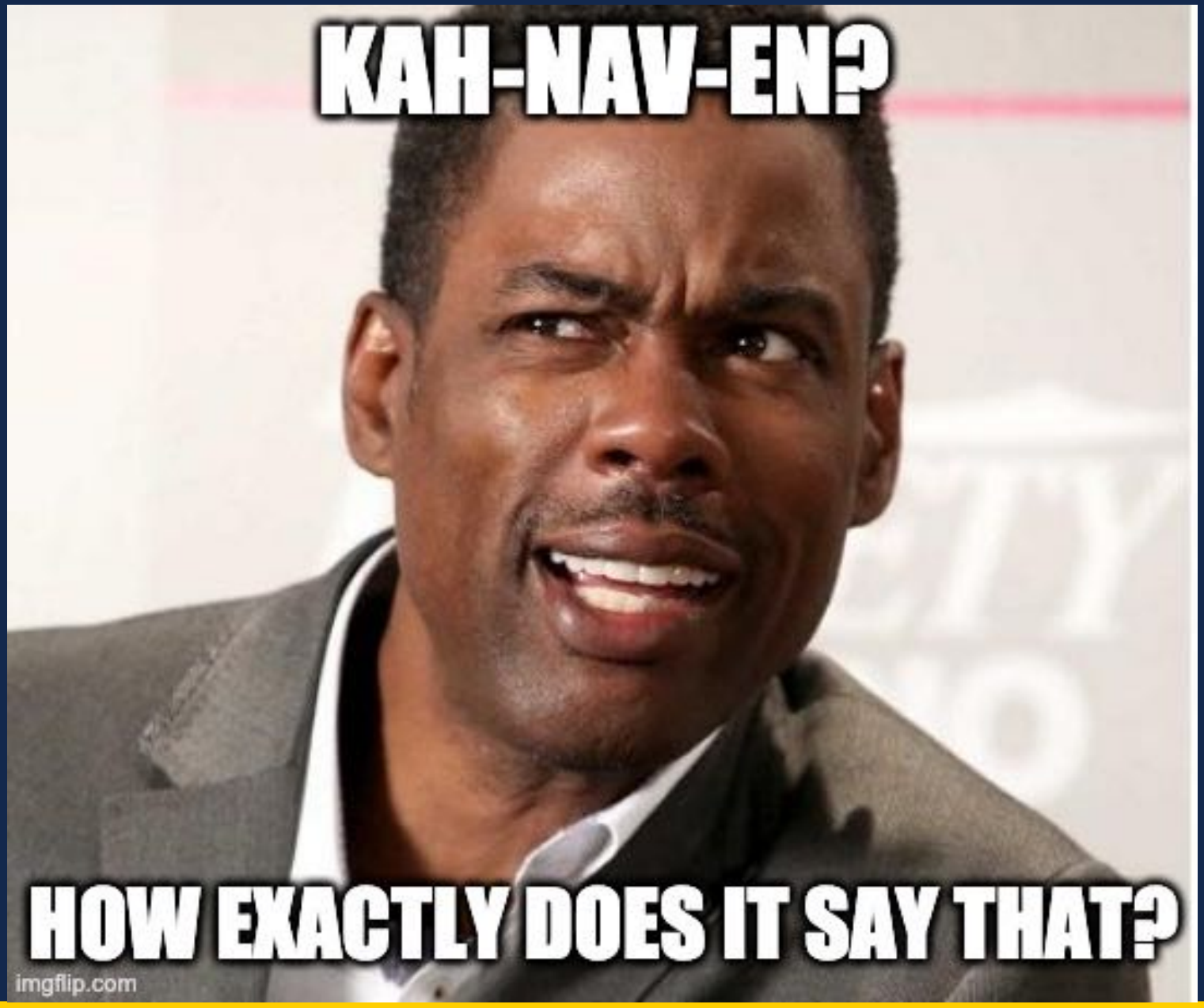
Overview of the Landscape of Problems



https://en.wikipedia.org/wiki/Cynefin_framework

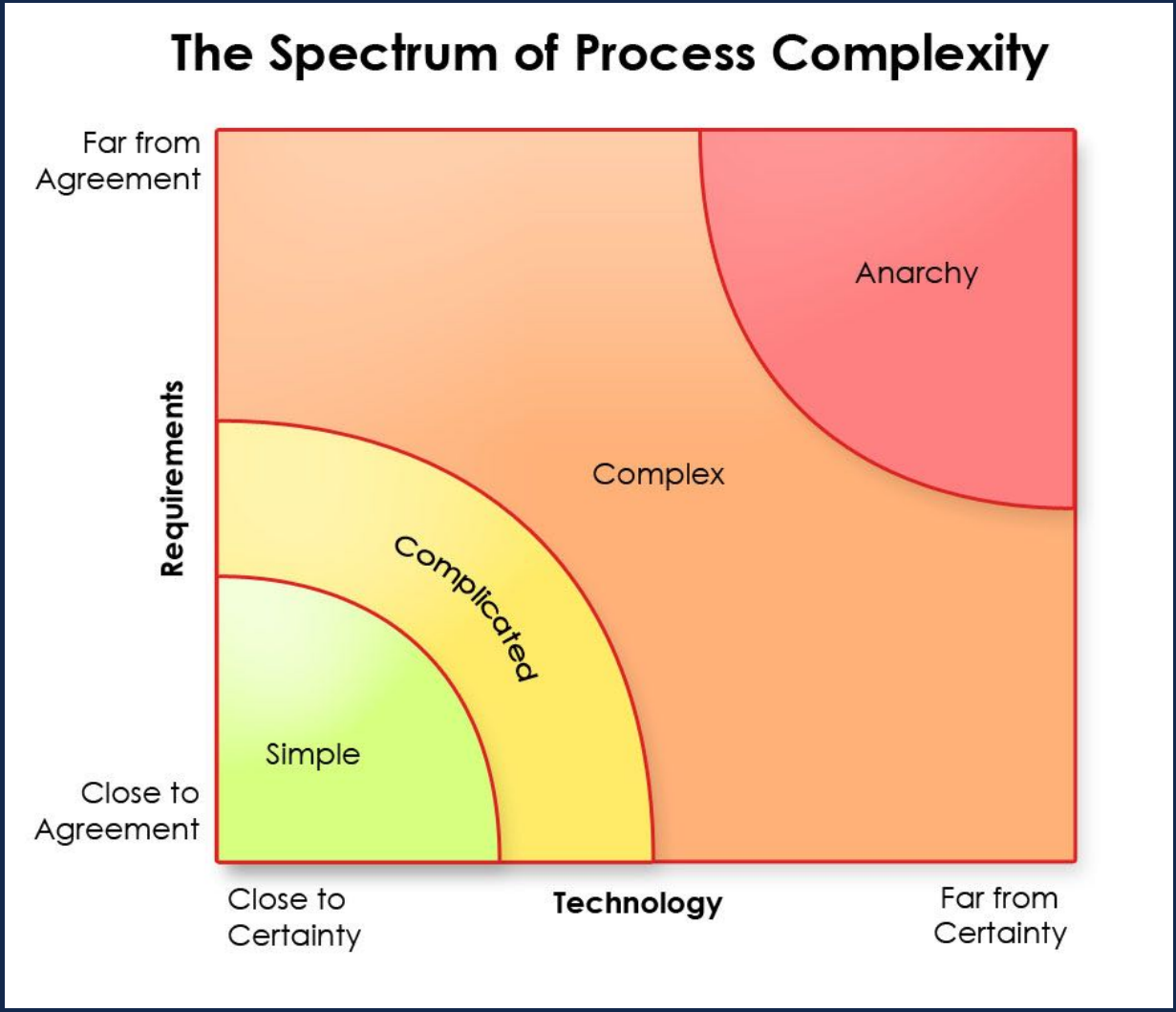
What is this thing?



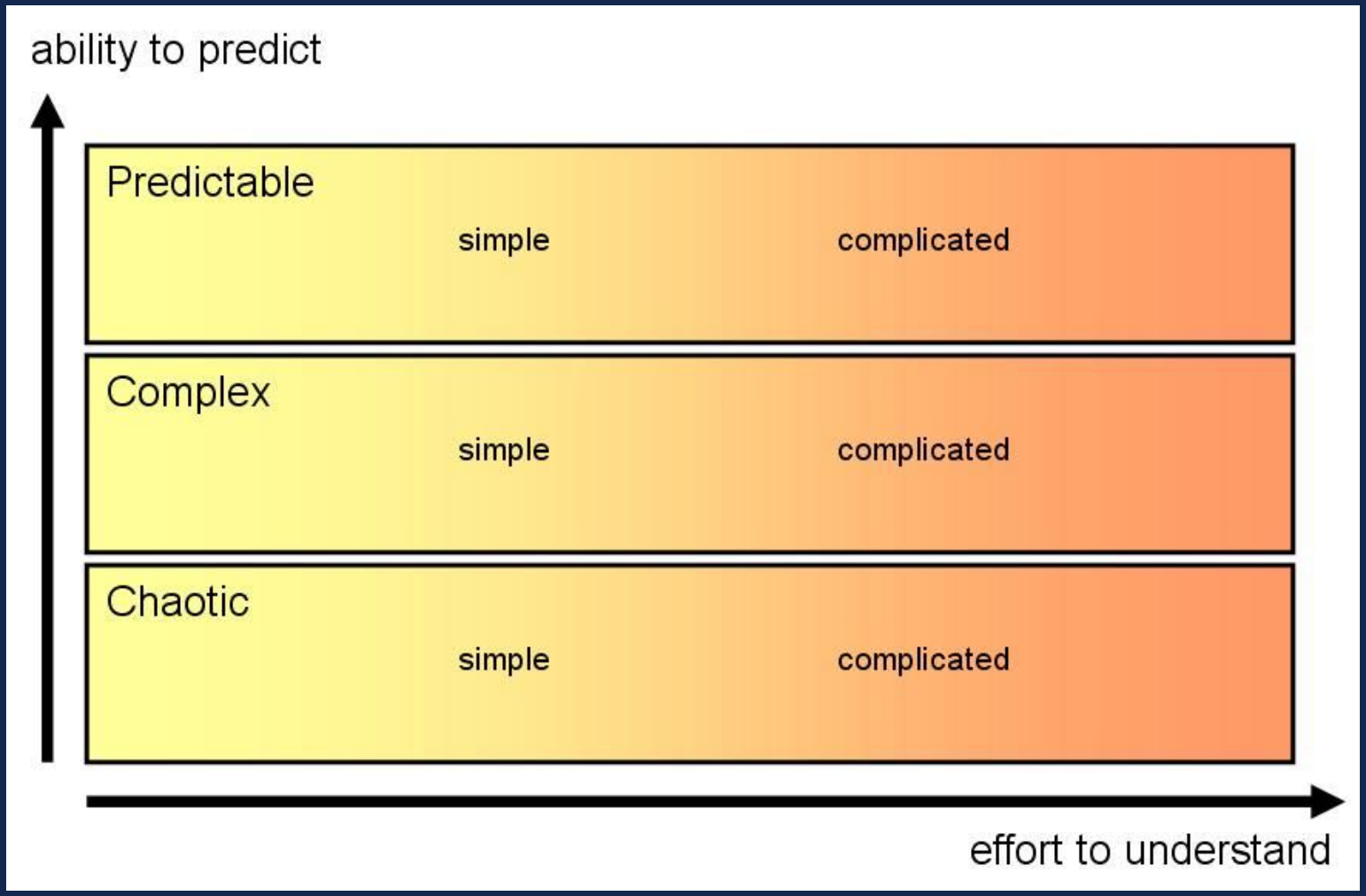


Project Reveal

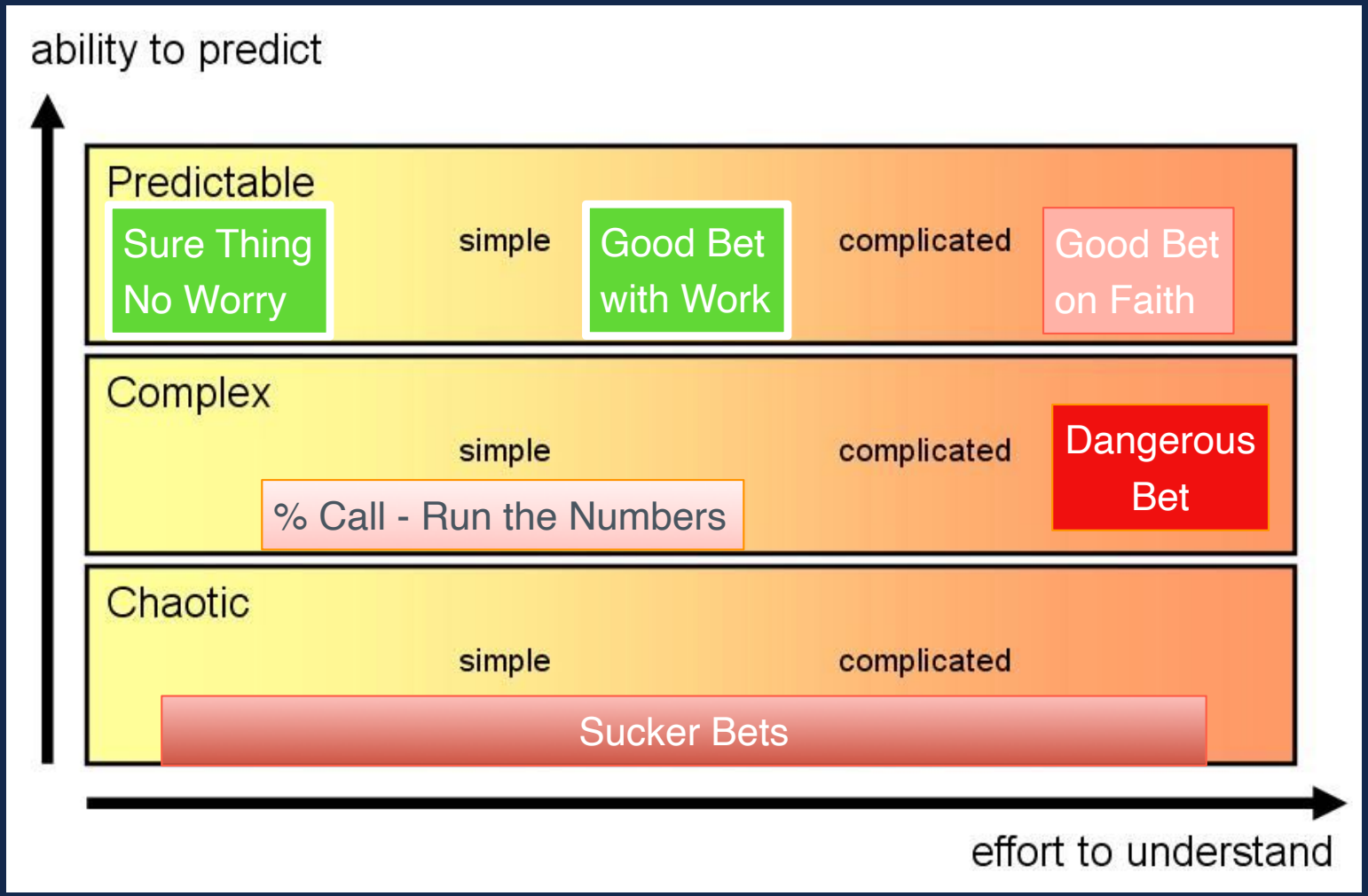
Now Move Down from Top Right to Bottom Left



Another View of the Continuum



If you had to bet



Categorizing This

Following a Recipe	A Rocket to the Moon	Raising a Child
SIMPLE (Puzzle)	**COMPLICATED** (Problem)	**COMPLEX** (Mess)
• The recipe is essential • Recipes are tested to assure replicability of later efforts • No particular expertise; knowing how to cook increases success • Recipe notes the quantity and nature of "parts" needed • Recipes produce standard products • Certainty of same results every time	• Formulae are critical and necessary • Sending one rocket increases assurance that next will be ok • High level of expertise in many specialized fields + coordination • Separate into parts and then coordinate • Rockets similar in critical ways • High degree of certainty of outcome	• Formulae have only a limited application • Raising one child gives no assurance of success with the next • Expertise can help but it is not sufficient; relationships are key • Can't separate parts from the whole • Every child is unique • Uncertainty of outcome remains

Source: ODI presentation, Exploring the science and complexity of aid policy and practice, London, 09 July 2008

As Coding

Tutorial <——-> PA

Building Something Known

Basic <———————> Adv

Making something new

SIMPLE (Puzzle)	COMPLICATED (Problem)	COMPLEX (Mess)
• The recipe is essential • Recipes are tested to assure replicability of later efforts • No particular expertise; knowing how to cook increases success • Recipe notes the quantity and nature of "parts" needed • Recipes produce standard products • Certainty of same results every time	• Formulae are critical and necessary • Sending one rocket increases assurance that next will be ok • High level of expertise in many specialized fields + coordination • Separate into parts and then coordinate • Rockets similar in critical ways • High degree of certainty of outcome	• Formulae have only a limited application • Raising one child gives no assurance of success with the next • Expertise can help but it is not sufficient; relationships are key • Can't separate parts from the whole • Every child is unique • Uncertainty of outcome remains

Source: ODI presentation, Exploring the science and complexity of aid policy and practice, London, 09 July 2008

Practically Speaking

Tutorial <——-> PA

Building Something Known
Basic <———————> Adv

Making something new

SIMPLE (Puzzle)	COMPLICATED (Problem)	COMPLEX (Mess)
11, 30, 100	110 C/Bs —-> 110/A	112 and Industry

Source: ODI presentation, Exploring the science and complexity of aid policy and practice, London, 09 July 2008

Hopefully this isn't you right now?

It shouldn't be…listen to folks who know

> Building software is incredibly hard, and building a culture of digital innovation is even harder.
>
> Ask Your Developer
>
> Jeff Lawson

So Again With Pix and A Clarifying Path

Phases of SE Academic Growth

Very Constrained

PAs

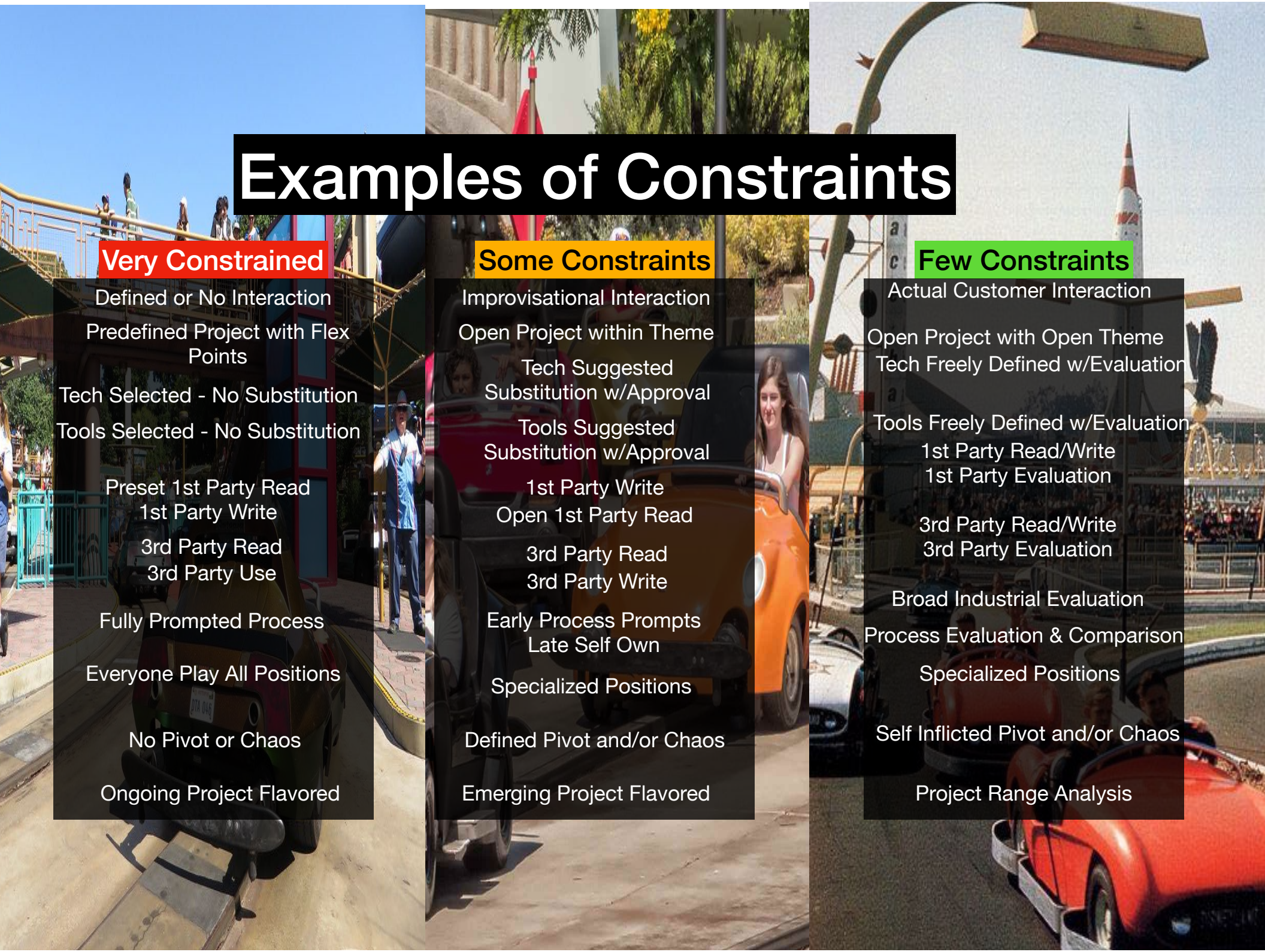
Some Constraints

110

Few Constraints

112, Your projects, special course

Examples of Constraints



Very Constrained

Defined or No Interaction

Predefined Project with Flex Points

Tech Selected - No Substitution

Tools Selected - No Substitution

Preset 1st Party Read
1st Party Write

3rd Party Read
3rd Party Use

Fully Prompted Process

Everyone Play All Positions

No Pivot or Chaos

Ongoing Project Flavored

Some Constraints

Improvisational Interaction

Open Project within Theme

Tech Suggested Substitution w/Approval

Tools Suggested Substitution w/Approval

1st Party Write
Open 1st Party Read

3rd Party Read
3rd Party Write

Early Process Prompts
Late Self Own

Specialized Positions

Defined Pivot and/or Chaos

Emerging Project Flavored

Few Constraints

Actual Customer Interaction

Open Project with Open Theme
Tech Freely Defined w/Evaluation

Tools Freely Defined w/Evaluation
1st Party Read/Write
1st Party Evaluation

3rd Party Read/Write
3rd Party Evaluation

Broad Industrial Evaluation

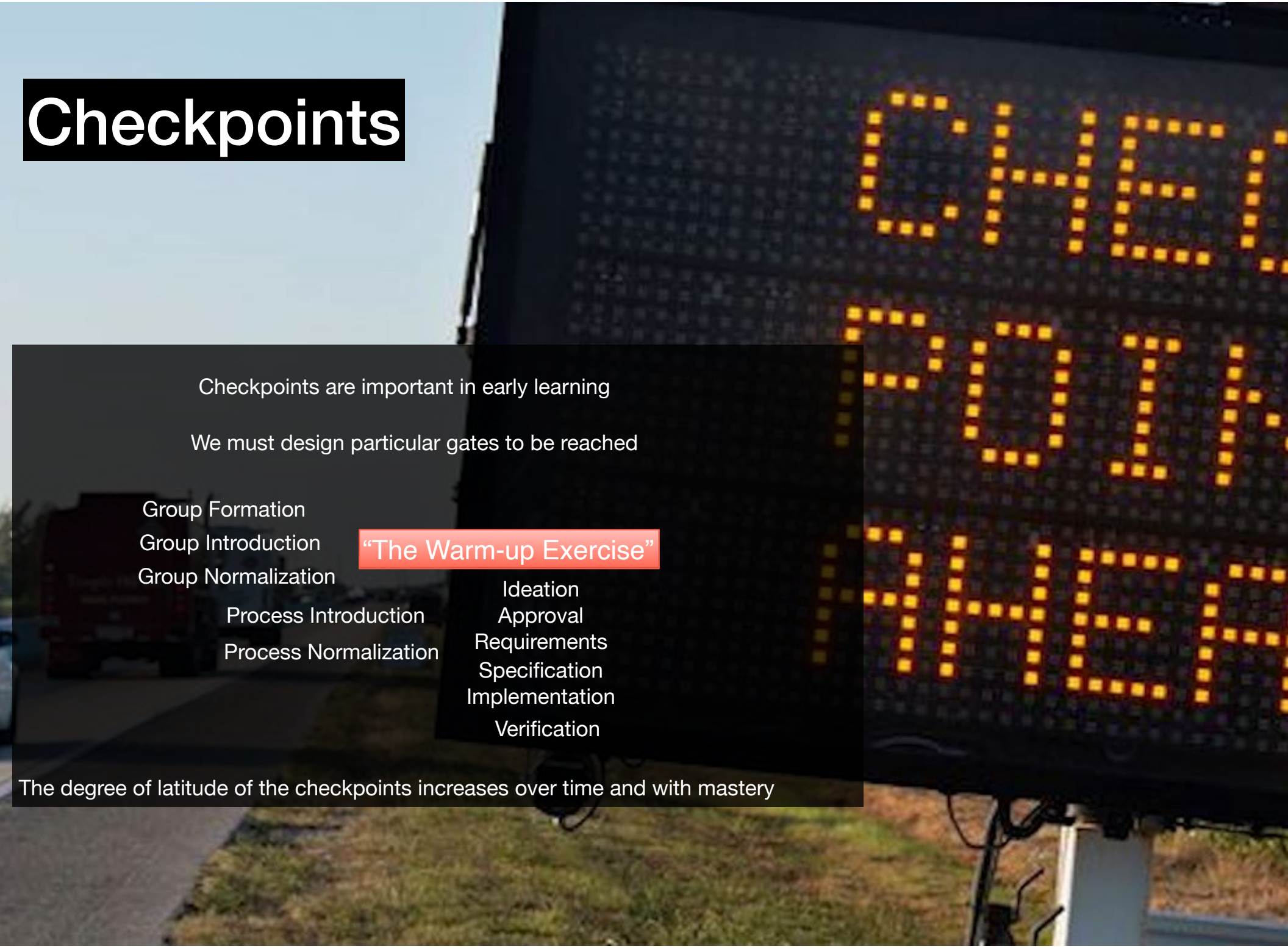
Process Evaluation & Comparison

Specialized Positions

Self Inflicted Pivot and/or Chaos

Project Range Analysis

Checkpoints



Checkpoints are important in early learning

We must design particular gates to be reached

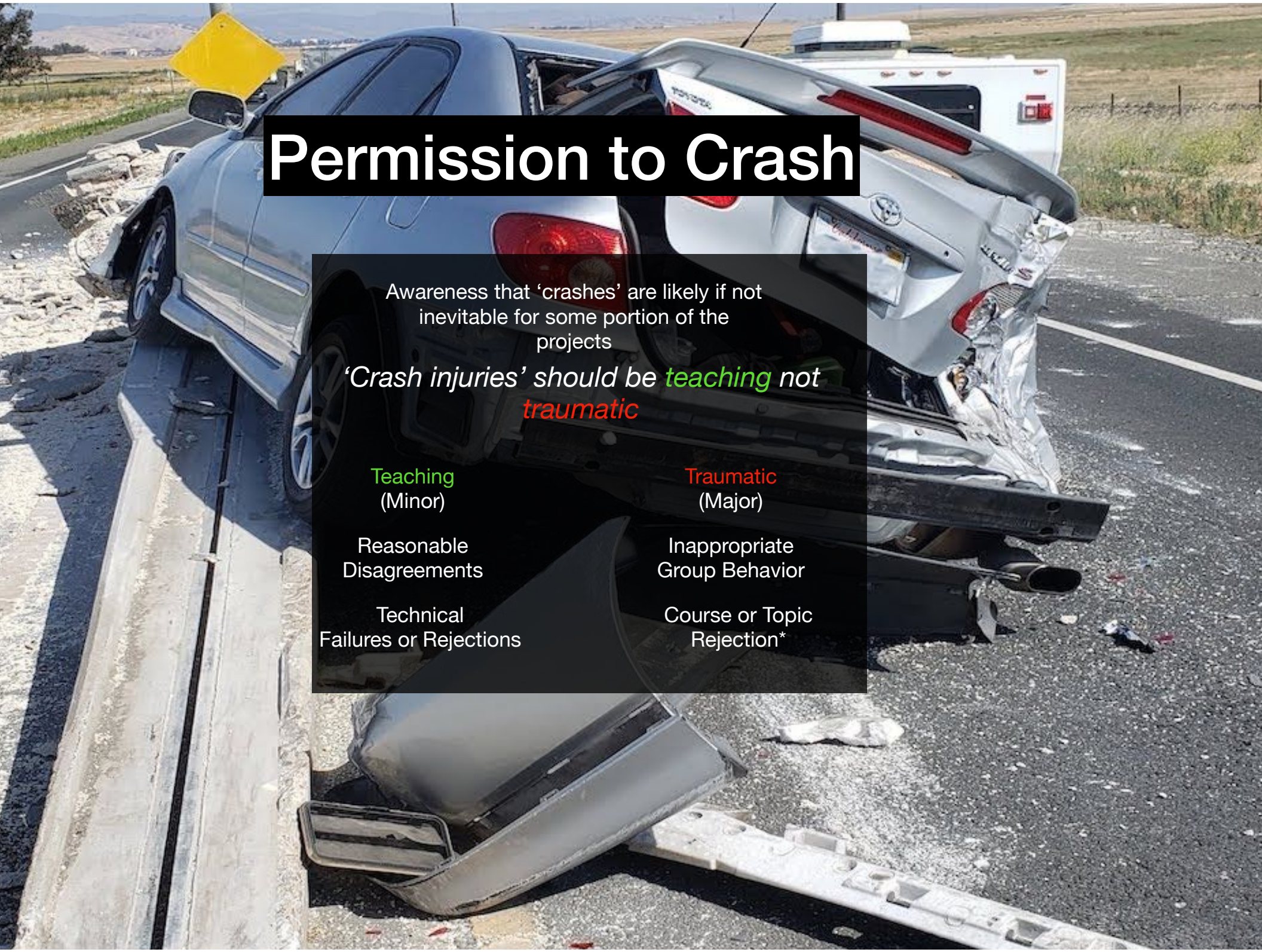
Group Formation
Group Introduction
Group Normalization

"The Warm-up Exercise"

Process Introduction
Process Normalization

Ideation
Approval
Requirements
Specification
Implementation
Verification

The degree of latitude of the checkpoints increases over time and with mastery



Permission to Crash

Awareness that ‘crashes’ are likely if not inevitable for some portion of the projects

‘Crash injuries’ should be teaching not traumatic

Teaching (Minor)	Traumatic (Major)
Reasonable Disagreements	Inappropriate Group Behavior
Technical Failures or Rejections	Course or Topic Rejection*

Preview - Where and How to Start?

Repo org, filenames, code organization (modules, function, etc.), code style

Systems and Tools To Manage + Process to Guide Us

Ok, ok “seatbelts” are on and we got a sense of the “trip”…so what is the project?

It will be a type of CRUD App because…

> The truth is that most software is pretty simple. It's what developers call CRUD applications: Create, Read, Update, Delete.
>
> ...
>
> Nearly every website or mobile app you've ever used is 96 percent CRUD operations. This isn't rocket science.

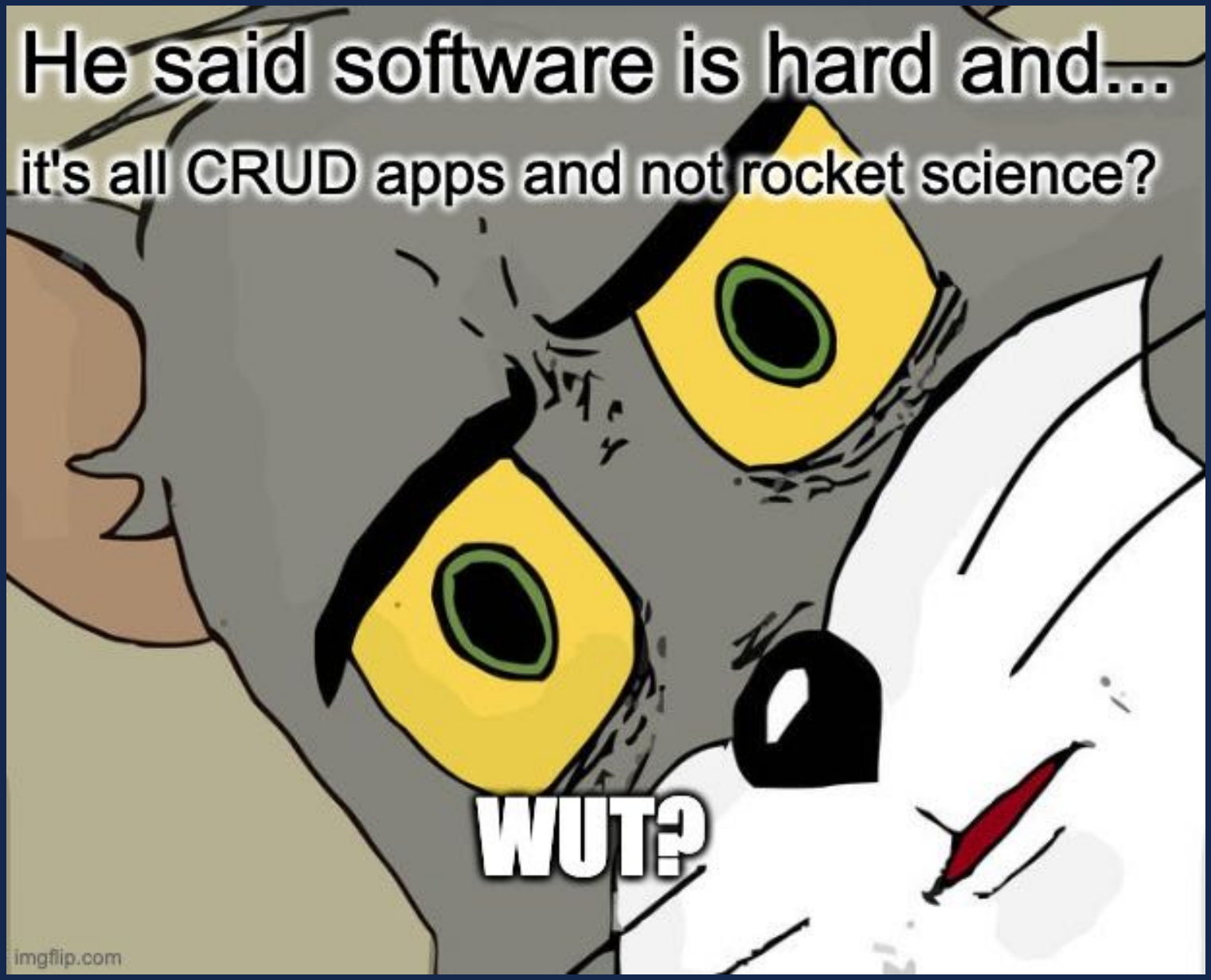
Ask Your Developer

Jeff Lawson

Meme Translation

CRUD Pattern

- **CRUD** = **C**reate **R**ead **U**pdate **D**elete
- Pattern of the actions we might perform on a resource or piece of data be that data in a db, memory, or on a file system
  - Ex: REST pattern - POST/Create, GET/Read, PUT/Update, DELETE/Delete
- A conceptually simple idea at the heart of apps.
  - Think a table or data grid with an add button, listing/ searching/paging, edit button, and delete icon - that's CRUD!
  - What's in the "table/grid" and how you work with it … now that's a bit more interesting!

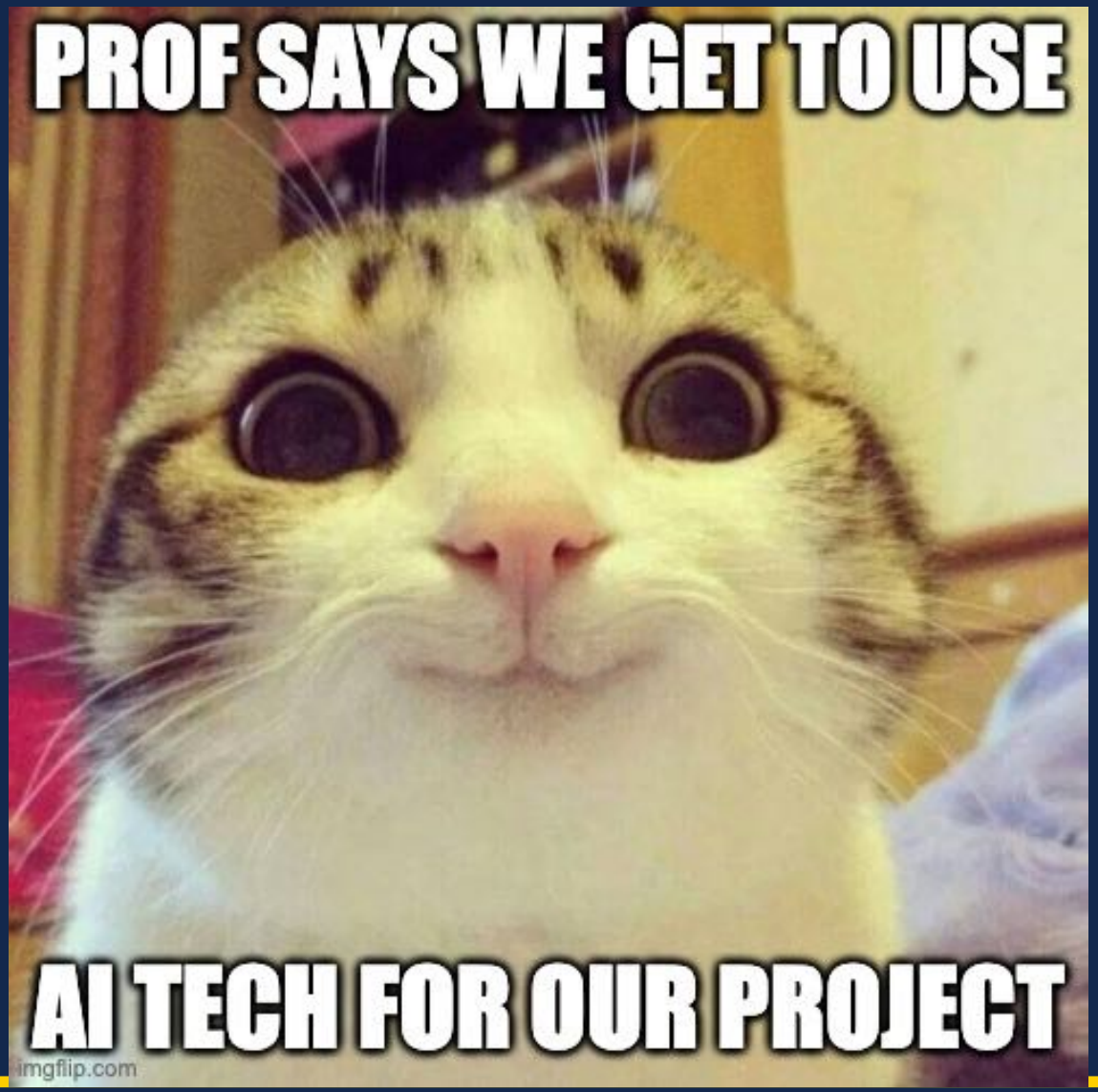


Project Reveal

Domain + Humans = The Problem

- CRUD itself is not a hard concept, the code is as he says not rocket science yet the problem is hard despite that.
- The problem and complexity he alludes to comes from two things
    1. **The Domain** - The details of what the CRUD does the rules of the domain, the jargon, etc. is not a given. In fact those small pieces matters more than anything
    2. **The Humans** - Both those you get those details from and you/team are not perfect in structure, belief or communication. What challenges humans introduce make things really hard!

So a CRUD App - Yet We Embrace the Hype!?!



Less CRUD more AI?

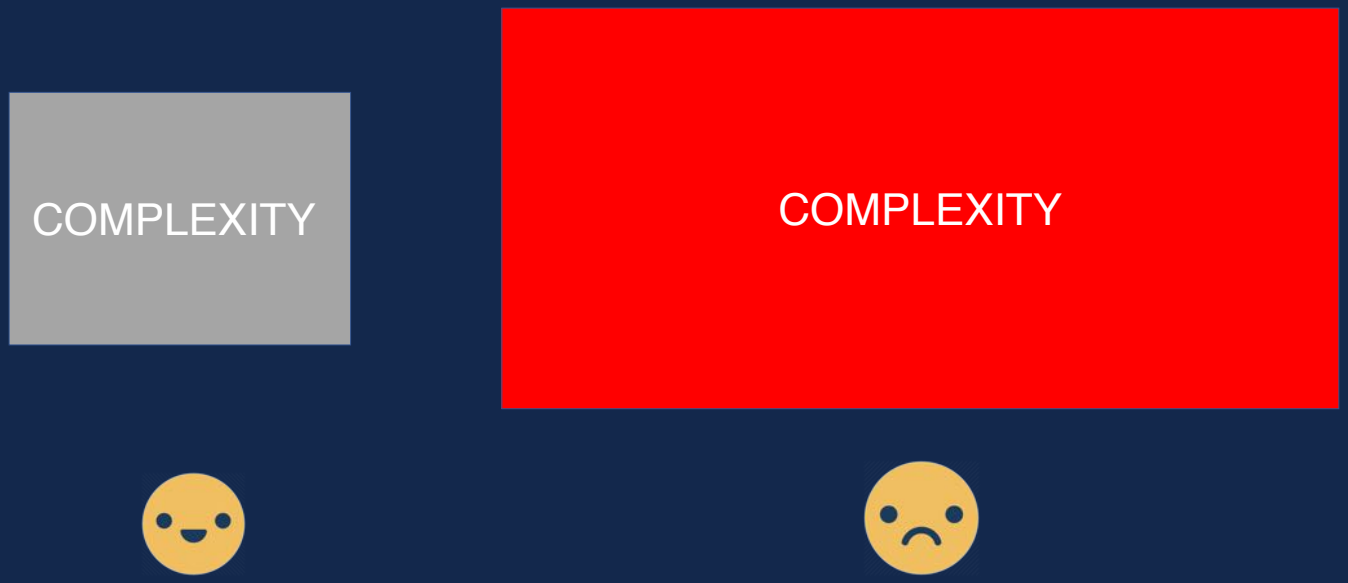
- Maybe, maybe not
- How would I do settings is that CRUD?
  - Themes, Languages, Features, etc.
- How much could I make of a simple journal app for tracking my work efforts with AI? Would it be a CRUD app no matter how the AI “solved” it?
- Spoiler as much as you want to get away from CRUD and other simple Input-Output apps you’ll see most things are indeed just that!

Flashback - Complicated and Complex

- Complicated things can knowingly be solved with rules, processes and patterns. They are hard, but manageable.
  - Simple apps such as CRUD functionally should be just at worst complicated, not complex
- Complex things are worse than complicated things, there is much unknown and how to solve them is unclear. Rules and patterns may not only let us down, but harm us causing us not to solve the problem the way it should be solved. This is especially true if we actually don't really know the domain
  - All these apps can get complex. These apps integrate with humans with thoughts and processes that may be ill defined, volatile or not followed

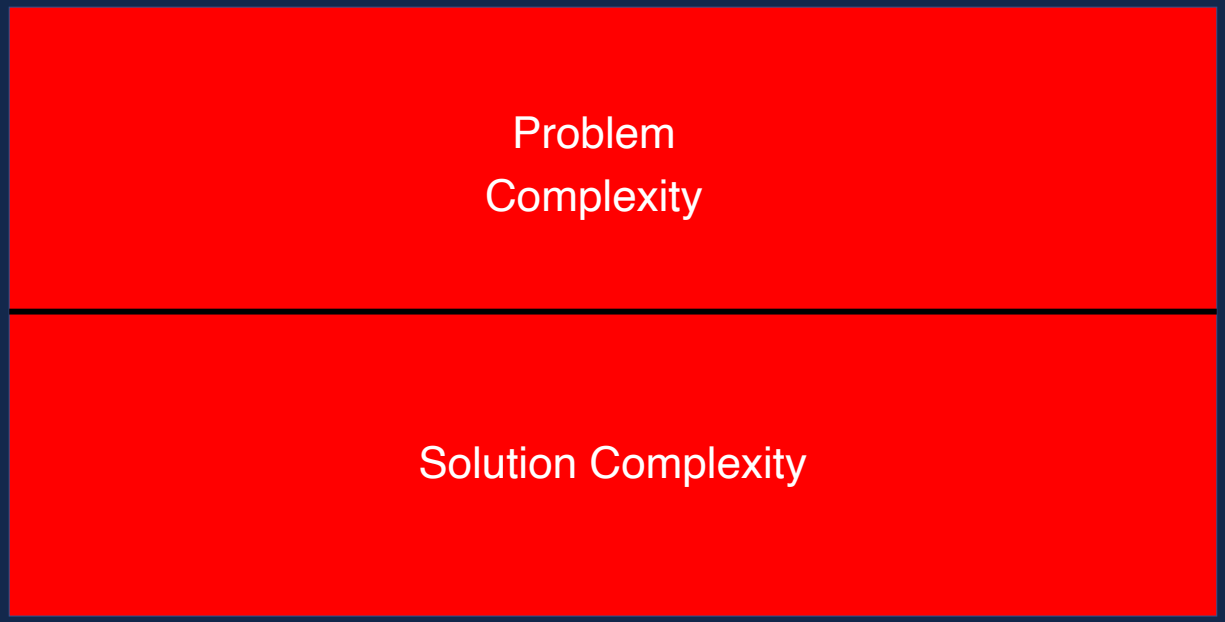
Complexity Redux

Complexity makes things hard and projects risky!

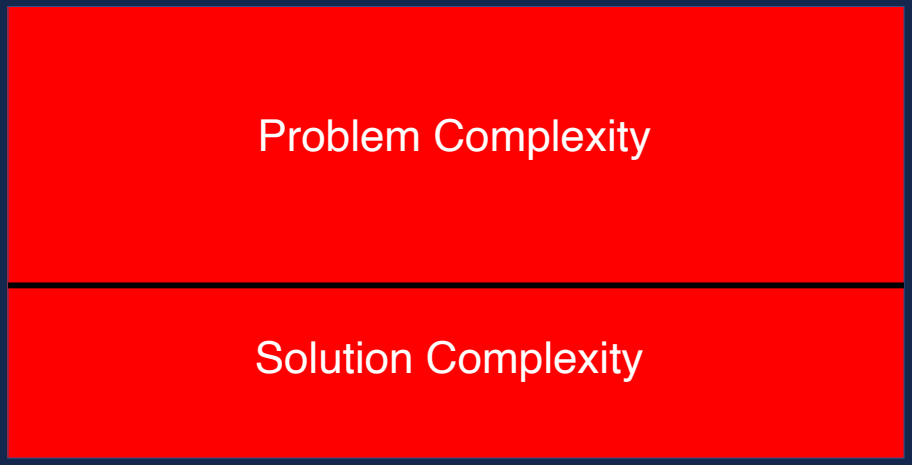


Obviously we want less of it if possible

In Some Fair World

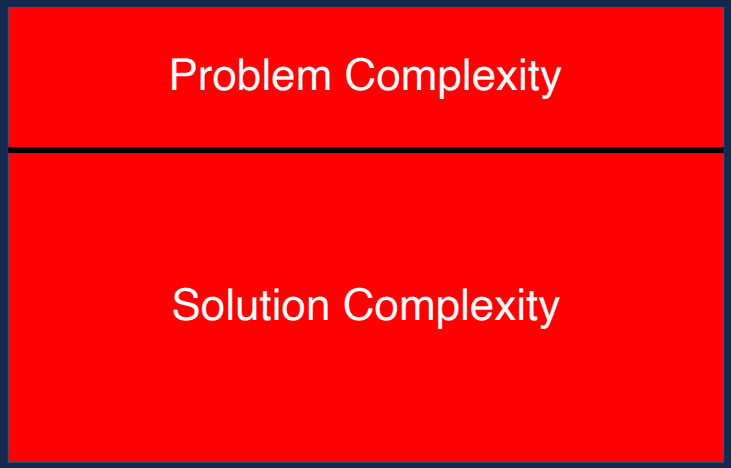


In Some More Idealized World



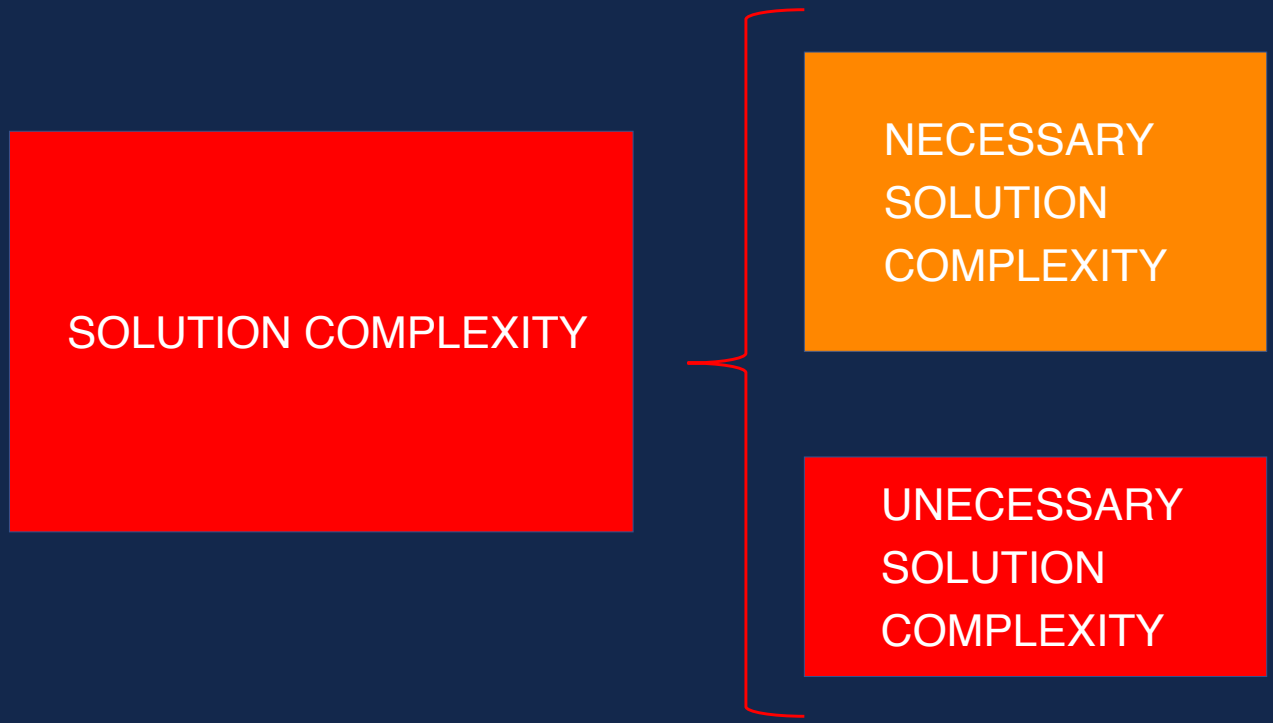
Problem is complex, but solution elegantly helps address it without expanding the complexity burden massively

Sadly Too Often

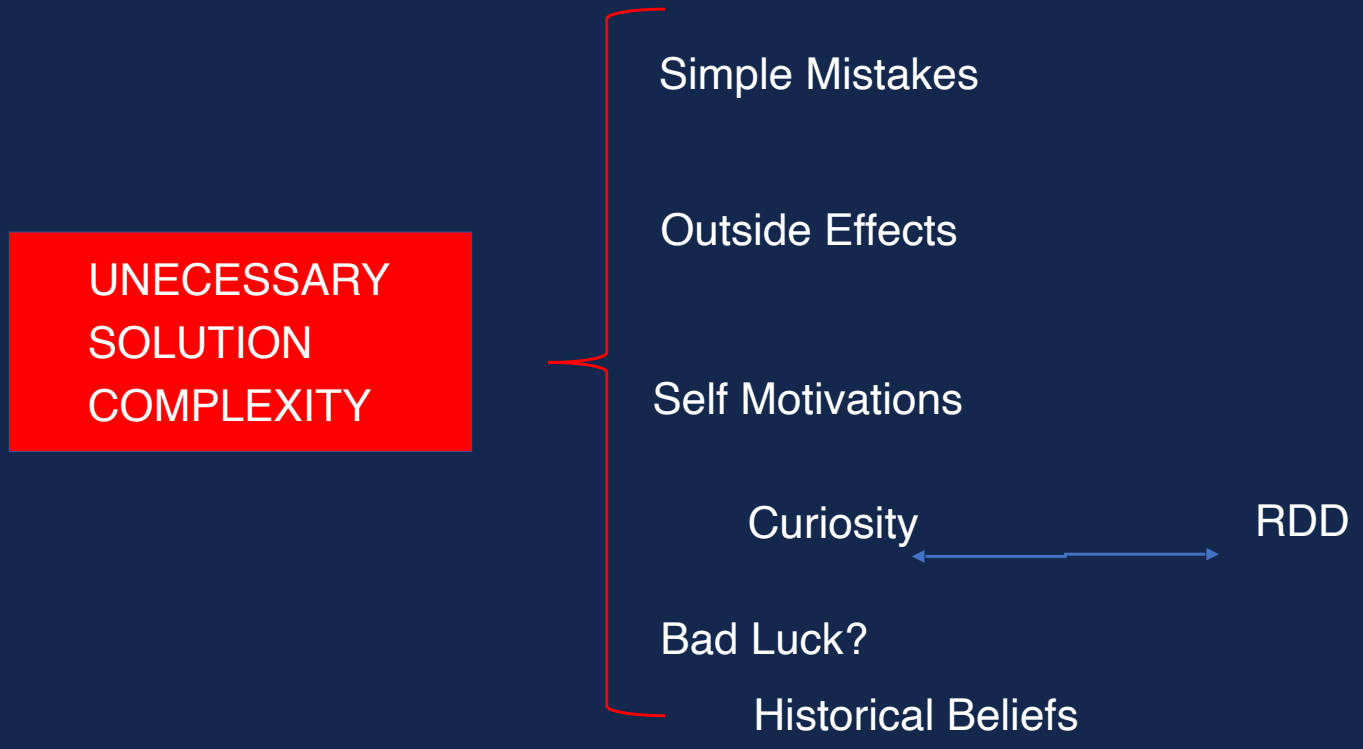


Problem is complex but solution adds lots of complexity

Why Solution Complexity?



Why Accidental Complexity?



Don’t Make Things Complex Unless Needed

- Good software makes hard things easy.
- Just because a problem looks difficult at first doesn’t mean the solution will have to be complex or hard to use.
- Too often, engineers go with reflex solutions that introduce undesirable complexity (*Let’s use ML! Let’s build an app! Let’s add blockchain!*) in situations where a far easier, though maybe less obvious, alternative is available.
- Before you write any code, make sure your solution of choice cannot be made any simpler.
- Approach everything from first principles.

Addressing Accidental Complexity

- More upfront thinking as opposed to just diving in
- Making sure the team member is as skilled as possible or has as much experience to avoid the simple mistakes
  - Can’t be fully solved for just minimized
  - In some situations you just have to go with who you have
- Strong Leadership and Good Team Trust
  - Acknowledge the need to learn, address the need to grow and build the resume, etc.
  - Section this off to areas for allowable risk or outside the project
    - Reframed Thoughts about 10% time at companies? Dual purposes maybe?

Addressing Inherent Complexity

- Some solution complexity is simply unavoidable
  - The complexity of the solution in some ways might be essential complexity as it is inherent to the problem itself (no way to avoid!)
- Addressing this type of complexity is only by moving back to the requirements and motivations of the project
  - If this complexity is very large better make sure it is worth it. Consider this a double check with the stack holder!

Complexity Summary

- Real Complexity
  Yes this is a hard problem and complexity is inevitable
- Accidental Complexity
  No the problem isn't truly hard, but complexity got introduced anyway
- When you have lots of features complexity is real (interaction and feature complexity)
- When you have been around for a while complexity often emerges (think backwards compatibility or environment changes)

Can Complex Be Made Just Complicated?

- Yes with enough time and effort (likely repetition required) we can break some complex problems down into the mere complicated thru patterns and encapsulation.

- *Prof's Opinion: This is less of our broad industry challenge. Sure this thought makes sense for truly "wicked problems" (ex. Autonomous Driving), but is not what most devs do day to day. This looks to me like using a corner case to justify complexity for what most people do…*

Can Complex Be Made Just Complicated?

…mostly, I see that we take things that are merely complicated and make them needlessly complex because we don't rigorously apply structure and do the work as we should. Other issues have to do with poor thinking, adding things in or following patterns, relying more on the perceived success of others as a heuristic instead of careful thought of the faced problem.

In short, our lack of calm, measured thinking, and discipline often results in a project-killing complexity being baked in.

"Project death often happening later so no learning either…"

Ok so warning heeded, then what are we doing?

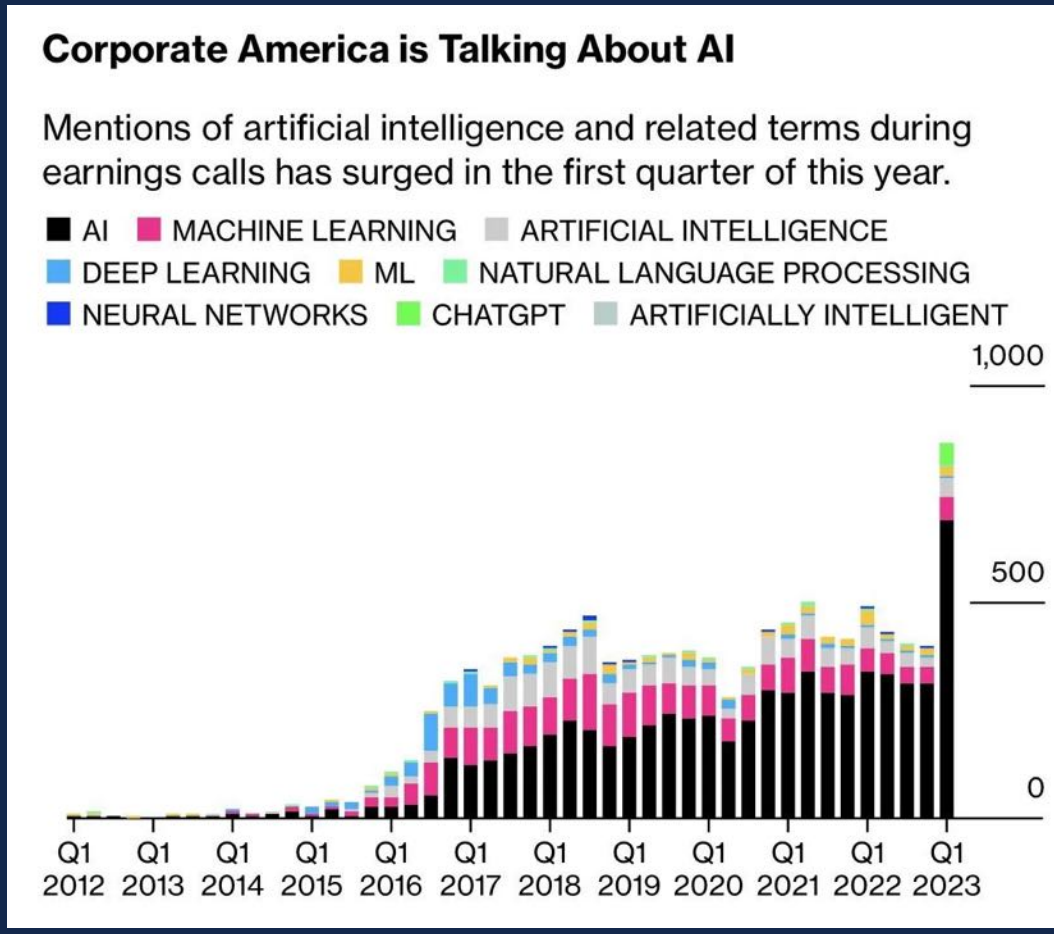
First, More On Our Current State of Affairs

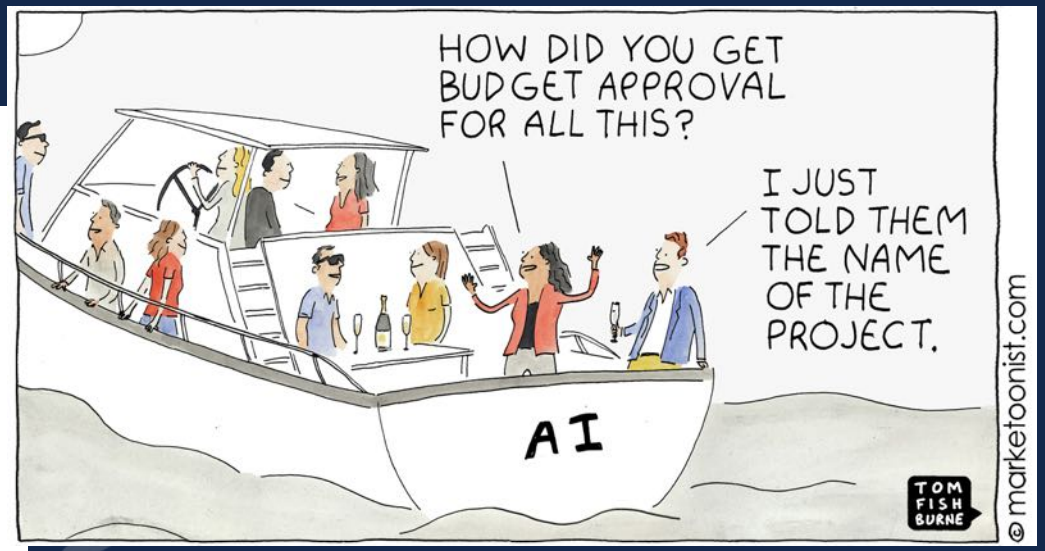
The Cloud Platform Era

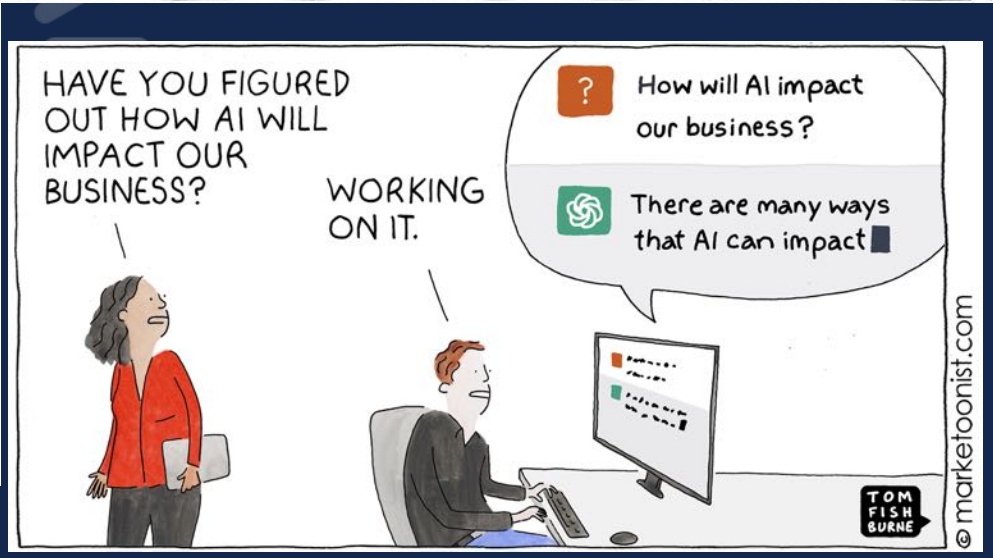
The Cloud isn't Always Great Nor Cheap

Sometimes these platforms go bad on is…see Twitter for example

Also…an AI Hype Tornado







A Modern Tragedy

An *incredible journey* is: One company buying another and closing its services down. -Our Incredible Journey

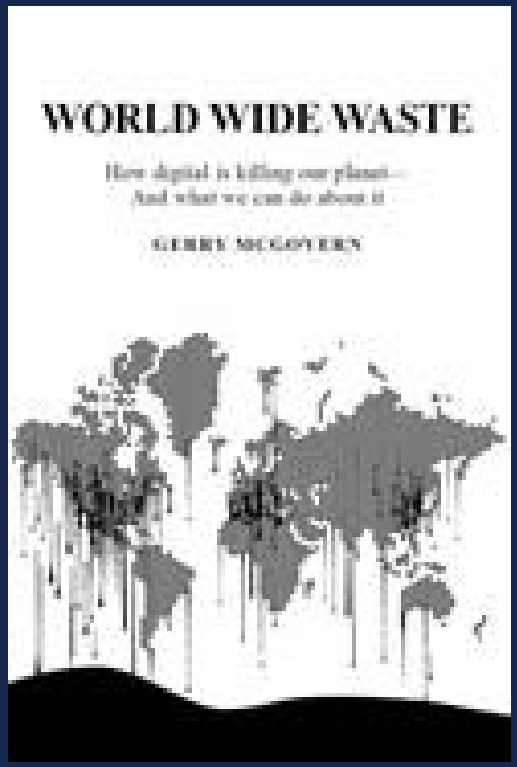
What Do People Now Expect?

Yep we get this !?!

All of the data you could ever want,
whenever you want it, at your finger tips

And lastly a world of disposability

- Code and data is not costless!
- Blasting things out with AI or npm installing means an explosion of things we don't need
- We don't have to look far to see it (cough cough our home page)
- Read some fun factoids here https://gerrymcgovern.com/books/world-wide-waste/



Are There Alternatives?

Local-First Software, Traditional Deterministic Approaches, Minimalism and Protocol Focus

What is Local-First Software?

- Local-First software resides on a user's device
- Local-first software does not reject the cloud, but changes the traditional relationship with it

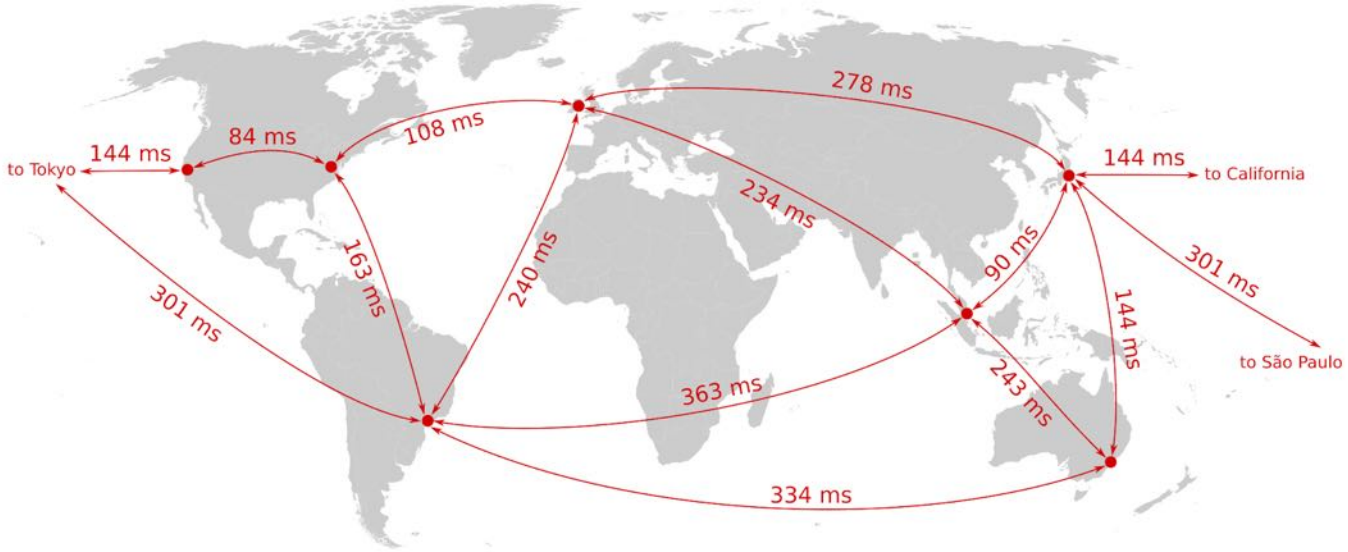
Useful Source: Ink & Switch

https://docs.google.com/presentation/d/1xqvntURBEwCR-zsS0PP1DD_hdibU8rNJVCevqQeLb0c/edit#slide=id.p

The Seven Ideals of Local-First Software

1. No Spinners: Your Work at Your Fingertips

- No waiting for data to finish loading from anywhere, it's already on your device

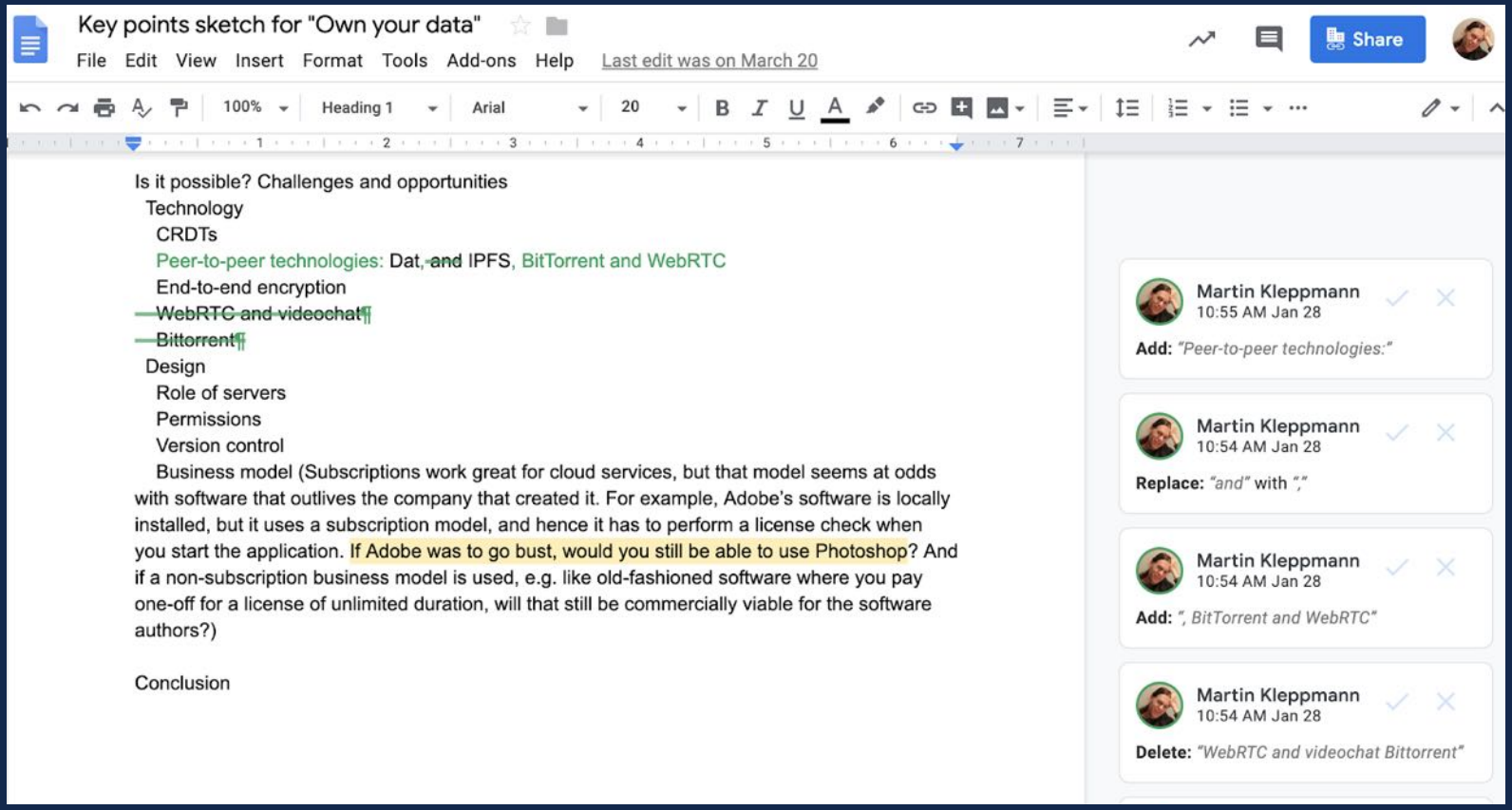


2. Your Work is Not Trapped on One Device

3. The Network is Optional

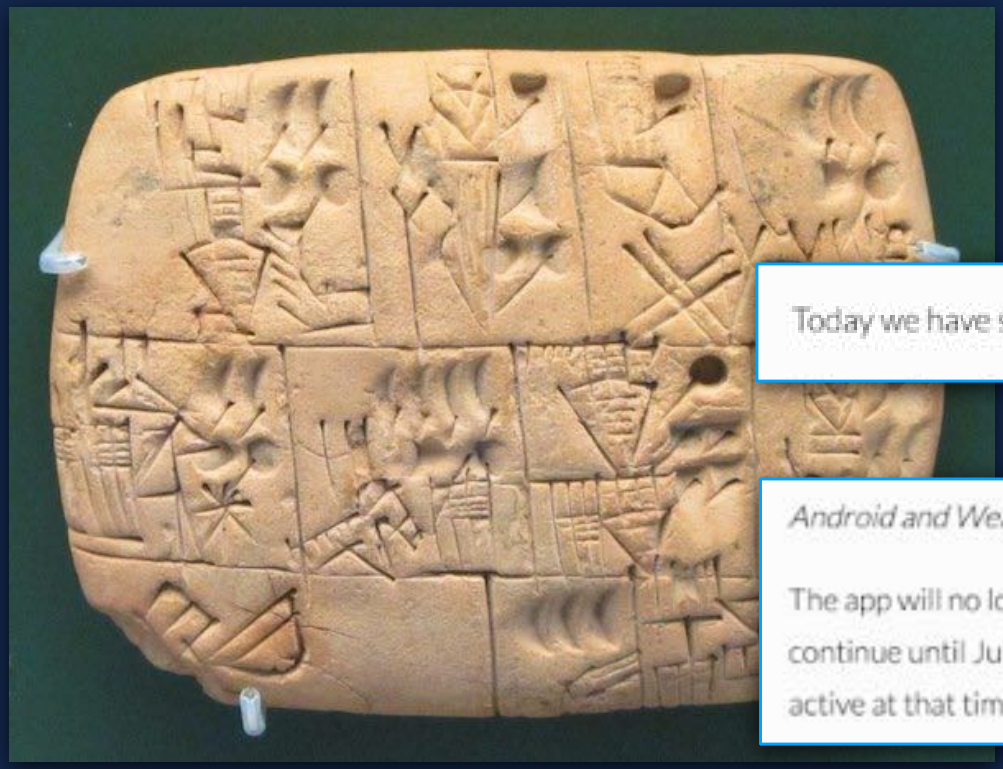
- Assume that your users will not always have access to the internet

4. Seamless Collaboration with Your Colleagues



5. The Long Now

- Design your products such that they will still work even after your support stops



We have some important news to share: Dark Sky has joined Apple.
You can read more on our blog.

App Maps Dark Sky API Help

Dark Sky Has a New Home

Today we have some important and exciting news to share: Dark Sky has joined Apple.

help as many people as we can stay dry and safe, and to do so in a way that respects your privacy.

There is no better place to accomplish these goals than at Apple. We're thrilled to have the

Mar 31, 2020
Dark Sky Has a New Home

Android and Wear OS App

The app will no longer be available for download. Service to existing users and subscribers will continue until July 1, 2020, at which point the app will be shut down. Subscribers who are still active at that time will receive a refund.

Website

6. Security and Privacy by Default

- Users have to *opt out* of privacy

7. Users Retain Ultimate Ownership and Control

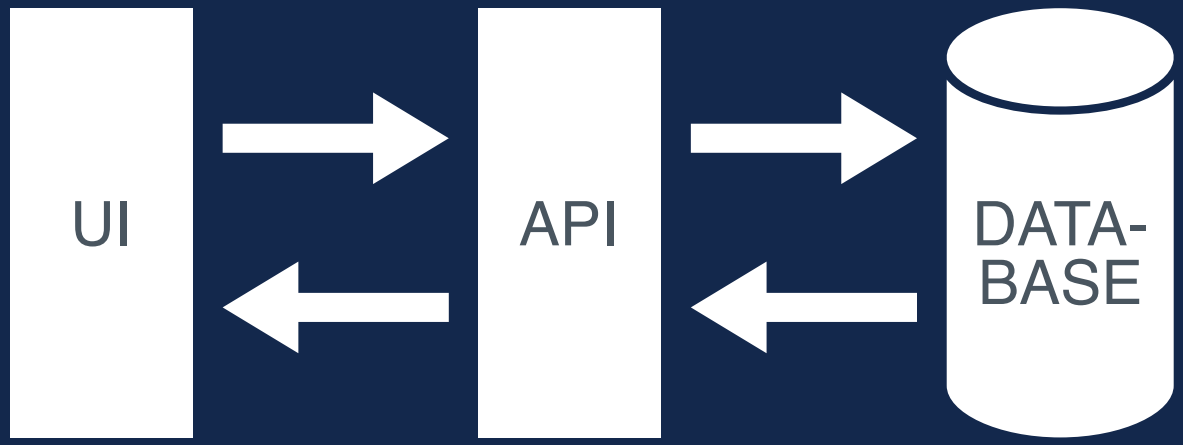
- Similar to the Long Now, users should own and control their apps & data. You should not be able to pull the plug on your project and take it from them.

The Developer's Case For Local-First

Cloud Software

- Expensive to operate
- Complex distributed system
- Multiple programming languages
- Difficult to scale

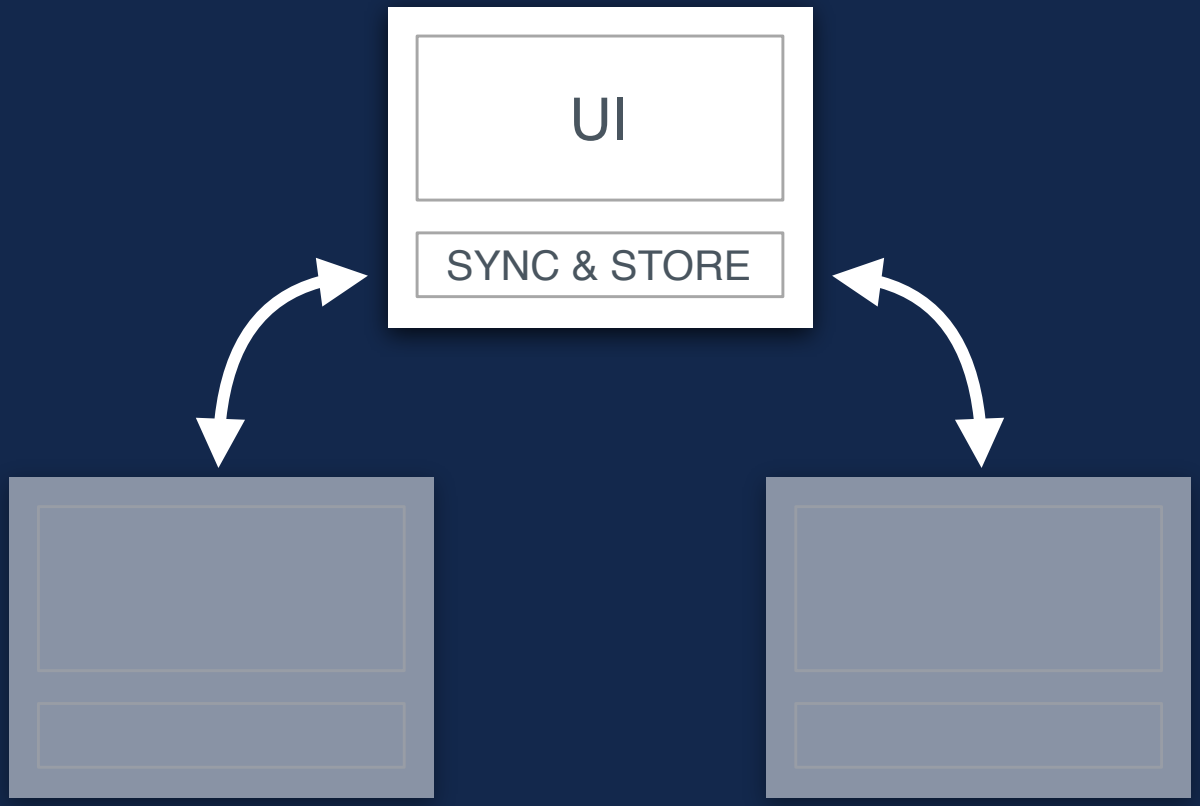
3-Tier Architecture



Self-Contained Software

- In order to support maximum capability while offline, most of the system's logic lives in one place.
- Benefits
  - Easier to reason about.
  - Architecturally simple.
  - Fewer codebases & languages.
  - Limited cloud infrastructure simplifies operations & scalability

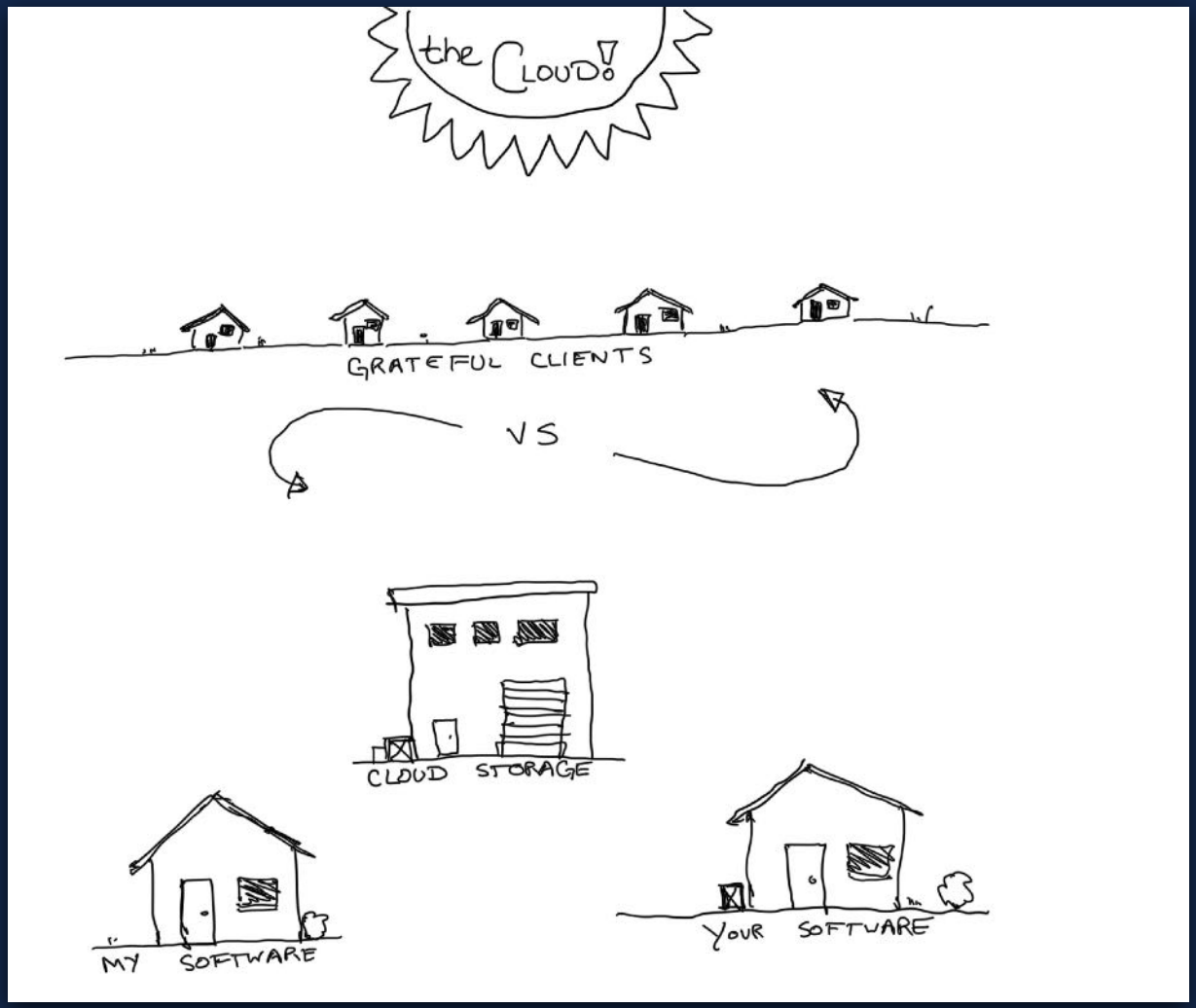
Self-Contained Software



Rethinking How We Relate to the Cloud

- We do not propose rejecting the cloud, but re-evaluating how we take advantage of it
- The cloud can be a helpful peer in a different role rather than the source of all truth
- Take advantage of the cloud to improve storage, availability, & compute power

Rethinking How We Relate to the Cloud



Software Minimalism

Maximal Development

- Too many software projects are turning into monstrosities of bytes and dependencies
- These projects are brittle (breaking with version updates) and we are subject to various “supply chain” attacks through dependencies.
- Maximalism has become popular for a few reasons
  - Do what the big folks do (see Microservices, React, etc.)
  - What happens if we scale thinking!
  - Complexity as a proxy for competency
  - No True Software Dev Fallacies
  - Dev Social Media / FOMO
  - RDD
  - Other ideas?

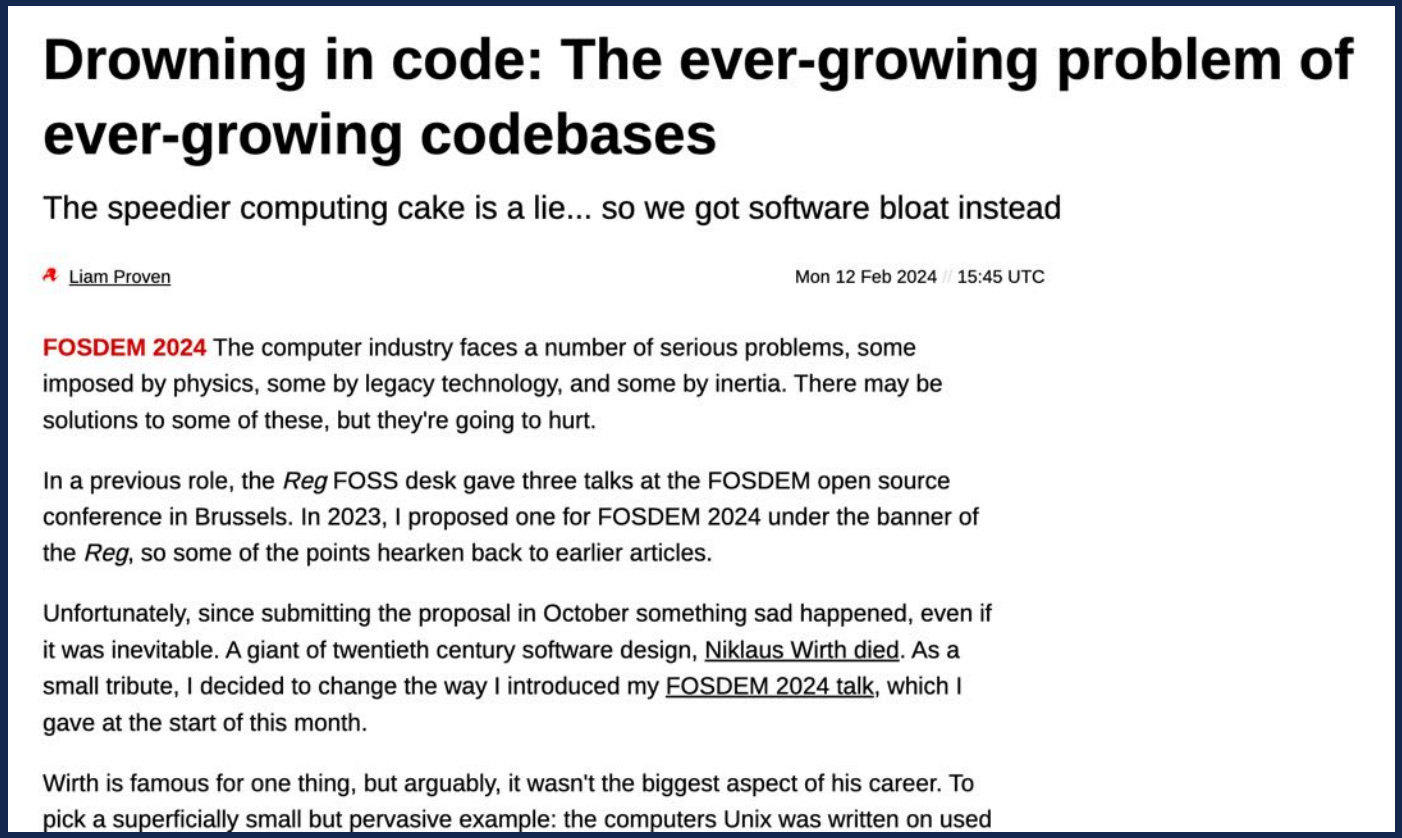
A Quote to Ponder

"To some, complexity equals power. (...) Increasingly, people seem to misinterpret complexity as sophistication, which is baffling—the incomprehensible should cause suspicion rather than admiration."

https://cr.yp.to/bib/1995/wirth.pdf

—Niklaus Wirth, 1995

Read a Take On This



Drowning in code: The ever-growing problem of ever-growing codebases

The speedier computing cake is a lie... so we got software bloat instead

Liam Proven Mon 12 Feb 2024 // 15:45 UTC

FOSDEM 2024 The computer industry faces a number of serious problems, some imposed by physics, some by legacy technology, and some by inertia. There may be solutions to some of these, but they're going to hurt.

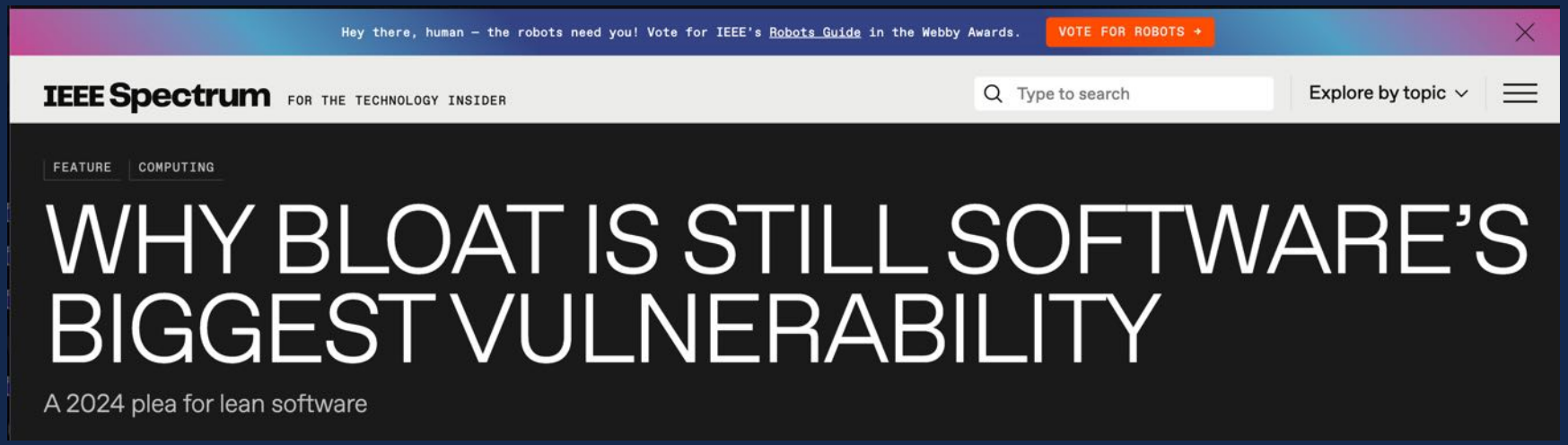
In a previous role, the *Reg* FOSS desk gave three talks at the FOSDEM open source conference in Brussels. In 2023, I proposed one for FOSDEM 2024 under the banner of the *Reg*, so some of the points hearken back to earlier articles.

Unfortunately, since submitting the proposal in October something sad happened, even if it was inevitable. A giant of twentieth century software design, Niklaus Wirth died. As a small tribute, I decided to change the way I introduced my FOSDEM 2024 talk, which I gave at the start of this month.

Wirth is famous for one thing, but arguably, it wasn't the biggest aspect of his career. To pick a superficially small but pervasive example: the computers Unix was written on used

https://www.theregister.com/2024/02/12/drowning_in_code/

Another Take (Many Exist)



https://spectrum.ieee.org/lean-software-development

The world ships too much code, most of it by third parties, sometimes unintended, most of it uninspected.

Return to Elegance and Minimalism

- In CS, we used to celebrate the small and elegant solution, so I propose we return to it.
- Minimalism has many benefits
  - Less = Longevity? Security? Better Maintenance? …
  - Less also means less power/resource impact
- Doing code this way tends to mean an improvement in quality and interestingly when look at design outside of computing it is rarely argued that more complicated == better

Protocols Not Platforms

Protocols and Platforms

- Protocols and standards like HTTP, HTML, Markdown, URLs, Git, etc. are open and portable
- Platforms like Instagram, Github, Medium, Twitter, etc. are not so open nor portable
- This lack of open nature is by design and creates a form of lock-in which helps preserve market position
  - This is wonderful when you are the dominant player and terrible if you are not

Protocols Return?

- An emphasis on protocols over platforms is pretty clear now as people see what happens when platforms such as Evernote, Twitter, Reddit, etc. degrade.
  - Often API access is removed, exporting made difficult, rules change, etc.
  - Cory Doctrow talks about this trend you might like to look up the somewhat salty term that is now becoming the way this process unfolds

- Some protocols like Markdown for notes (see Obsedian), ActivityPub for Social (see Mastodon and even Threads), Git for far beyond Github (see Gitlab and many more) are becoming more and more popular. Web frameworks are being seen for their overhead, and more platform-specific frameworks are getting much more popular.

We aim for protocols here and to abstract platforms if we decide to use them in certain places

Broad Specifics In Place!
What's Our Domain?

Cards! Cards! Cards!

Cards

- A rectangle
- Smallish object
- Front and Back
- Combination of text, symbols, and picture(s)
- Domain Specific Content
- Context-Specific Use
- Fun and Familiar to you?

A $420k PSA 10 Charizard Card!

Card Variety!

- Card Collector App
- Custom Card Maker App
- Flash Card Study App
- Business Card App
- Card Game App*
- Some other card thing**

Project Reveal

* - Danger!

** - TA Approval, though all need that really

//CSE 110;

Beware The Lure of the Game Path

- I have much experience with student projects being quite unsuccessful because of accidental complexity and scope naivete. In effect, self-inflicted grade risk because of an unrealistic view of the ability to meet project scope.
    - The card game approach **is a significant step up in complexity from the other ideas**. There are many gotchas here that you probably aren't considering yet. With the broad project limitations, you also may find it simply intractable.
- TL;DR - assume this must be proved feasible, and know the TAs will be very stringent on design approval to protect you more than limit you.

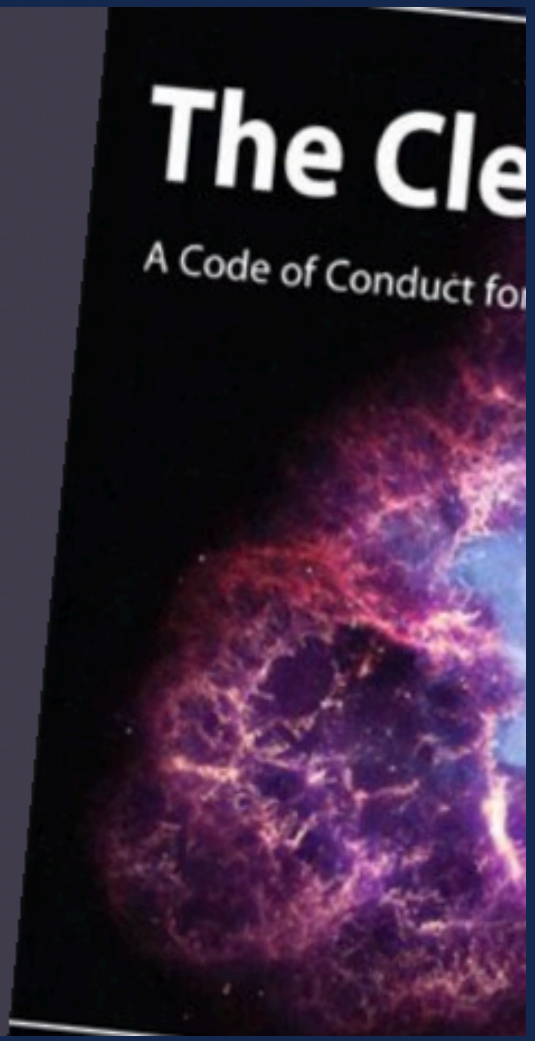
What No “Solutions” or Spec!?!

- All I did so far was to explain a concepts and prompt thought in you and your team.
- The slides before and after are demonstrative only and there are many other avenues to explore
- Your project will have to do this - YOU WILL CREATE THE DETAILED SPEC for the problem. Lots of latitude and many “solutions” you just can’t change the premise nor vary the tools / system.

Domains Matter - Don’t Screw Up on Step 1

> It is the responsibility of every software professional to understand the domain of the solutions they are programming.

The Clean Coder
Robert C. Martin



DDD and Others Say This Even At Code Level

Domain-driven design is predicated on the following goals:

- placing the project's primary focus on the core domain and domain logic;
- basing complex designs on a model of the domain;
- initiating a creative collaboration between technical and domain experts to iteratively refine a conceptual model that addresses particular domain problems.

Concepts of the model include:

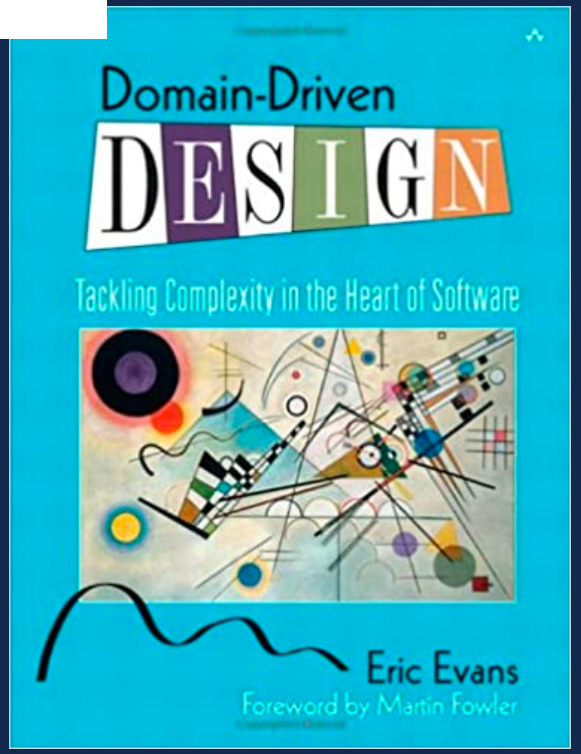
Context
The setting in which a word or statement appears that determines its meaning;

Domain
A sphere of knowledge (ontology), influence, or activity. The subject area to which the user applies a program is the domain of the software;

Model
A system of abstractions that describes selected aspects of a domain and can be used to solve problems related to that domain;

Ubiquitous Language
A language structured around the domain model and used by all team members to connect all the activities of the team with the software.

https://en.wikipedia.org/wiki/Domain-driven_design



Pro Tip - Spend Significant Time Studying the Problem and What It Solves for People BEFORE coding

Minimalistic, Open, and Expandable Local-First Web Tech Based Card App

What Does All of That Mean?

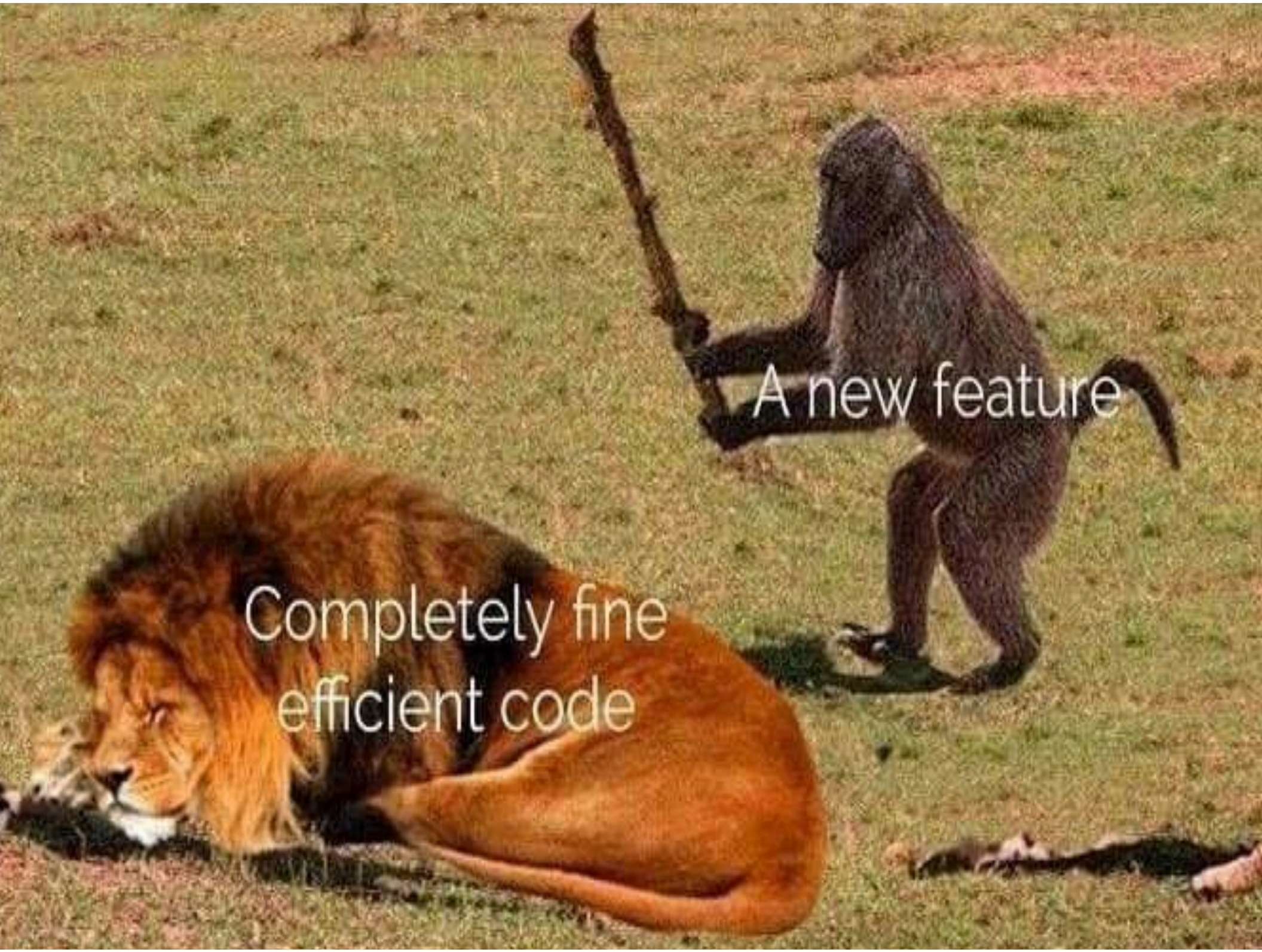
- Minimalistic means we use the raw web platform and related APIs and have as few dependencies as we can (robustness, maintainability)
- Open and Protocol Focused meaning that we make the system agnostic to future use by relying on some open format you design. Likely a form of JSON
- Expandable means that this app *could* be expanded and support other related content items or decks. For example, a collectors app might be specific to Pokemon but could have a way to be adopted to Yu Gi Oh someday.

What Does All of That Mean?

- Local-First meaning we use data that is saved on the device and potentially synced when convenient and don't go to the network unless needed (ex. eBay endpoint*)
- Web Tech Based HTML, CSS, JS, and media assets should be all we need. Frameworks and Meta frameworks not required, but we likely also don't need complex backends or distribution schemes. Note: This limitation doesn't preclude mobile or desktop apps!?!
- Card App meaning that the primary informational entity in the system is a "card" like a playing card, collector card (sport or not), flashcard, game card, etc.

*You will not be using networked services until you can demonstrate adequate mastery of local services. It would be great to see people getting to that step though!

List of features I've come up with for my future app
0.2 seconds after the project is announced

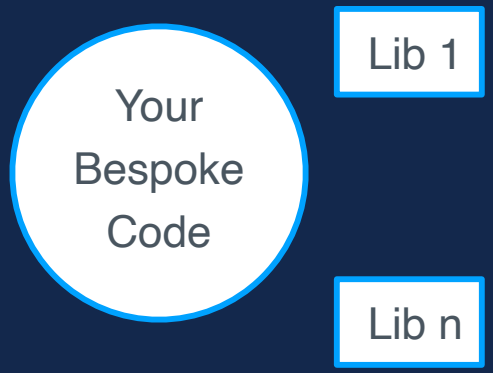

A new feature
Completely fine
efficient code

Allow Growth - Your App's Range Over Time

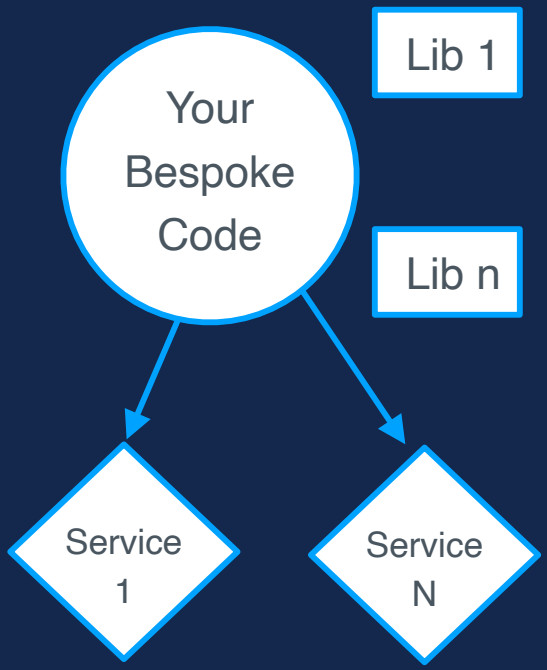
Start here

Maybe doing the warm-up!

Eventually discover your needs



Some day get here



You can of course plan for "some day" but don't let it add risk today!

Broadly Always Use Appropriate Materials and Set a Proper Foundation

What Does All of That Mean?

- Materials means the tech and tools we use. Less tools, more platform use, basically less moving parts = better over time (less to break, maintain, write, etc.)
- Appropriate means right tool/tech for the job. Sometimes you do need special materials or different approaches. You need to learn judgement to do that, but generally be simple at first and add complexity only when you need it. (Spoiler: YAGNI)
- Foundation means set the technology and design as a basis for other things to be built on. No need to do all the things yet, but you might plan so you could do those (ex. Localization, themes, voice, APIs online, etc.)

Tech Constraints Help!

- We start first with our core technologies
  - HTML, CSS, JavaScript, and common media formats like markdown, images, audio and video
  - While we are web tech that doesn't mean we are necessarily constrained to the browser and a site as our only delivery choice (ex. https://www.electronjs.org/, chrome apps, etc.)

- We should aim to be free of dependencies for core tasks and utilize carefully those for hard tasks as we encounter them.

- We are local first and remote second, later we can add 3rd party destinations (ChatGPT endpoints), but if we do that we should create an abstraction layer so that we can add as many as we like over time and future proof the design

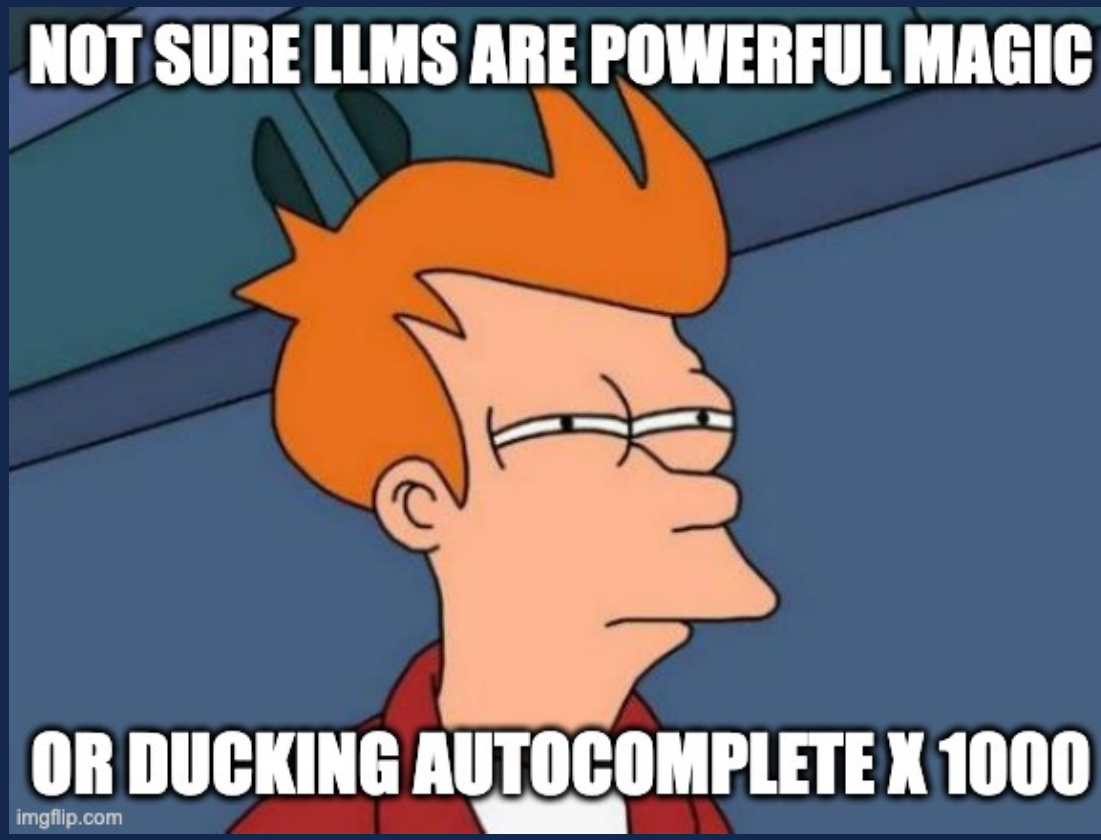
Warning: The constraints are here both for your protection and our need to reduce scope. Do not attempt to break them and TAs will write a negative signal if you do.

Keep your "workspace" clean, as lots of tools and dependencies makes work difficult!

Addressing LLMs and Other Generative AI Curve Balls

Magical Word Spitting Machines

- Generative AI is being hyped very very hard right now
- Positive - yes it is pretty impressive what you can do
- Negative - it makes sentency sentences, it's a b.s. engine, it answers plausibly not necessarily correctly



Natural Language vs Programming Language

In prompt engineering, instructions are written in natural languages, which are a lot more flexible than programming languages. This can make for a great user experience, but can lead to a pretty bad developer experience.

Programming Languages: More constrained, fewer synonyms, slang and idioms are less common but are existent, context free mostly (at least within code), syntax is more consistent

Natural Languages: Less constrained, lots of synonyms, slang and idioms very common, non-verbal or contextual data, sloppy or inconsistent syntax

Probabilistic Realities

- Prompts in LLMs are intrinsically not stable because of how they are implemented it is probabilities not if/thens if you like.
- Non stable = same prompt **may** return different answers
- The degree of stability can be influenced.
  - Open AI describes this as temperature - https://platform.openai.com/docs/api-reference/completions/create#completions/create-temperature

Hallucinations

- LLMs can and do appear to “make up” information
- Examples: links to non-existent sources, false statements, illogical statements, and just flat wrong information
- The term hallucination is given to such WRONG ANSWERS which is a form of marketing euphemism. This euphemism also is a bad idea because it imbues LLMs of having presence or mind which is a useful “frame” but is simply untrue.

Realistic Thoughts On Hallucinations

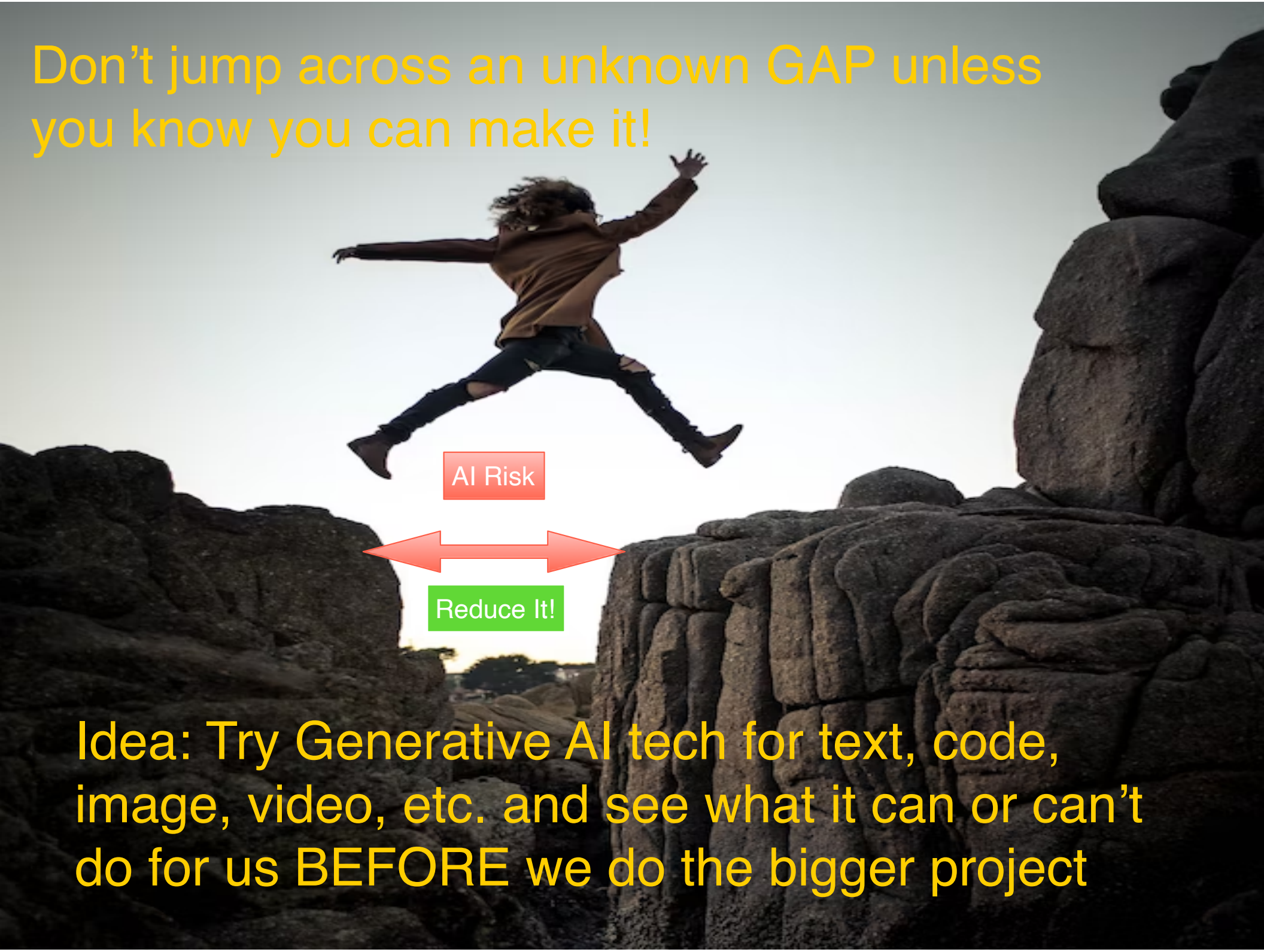
- LLM system fans are being deceptive or foolish if promoting a deterministic nature, encouraging perfect trust, or suggesting a presence of mind or understanding.
- A non-deterministic system that isn't perfect != not useful
- Consider our project prompt does we probably don't want our entries probalsitc, but if the code generation can be improved with an LLM we might like to use it. However, be careful just bolting on a "write with AI" feature because you can. Maybe use said features yourself and determine if they are actually valuable for you before thinking it is a good idea.

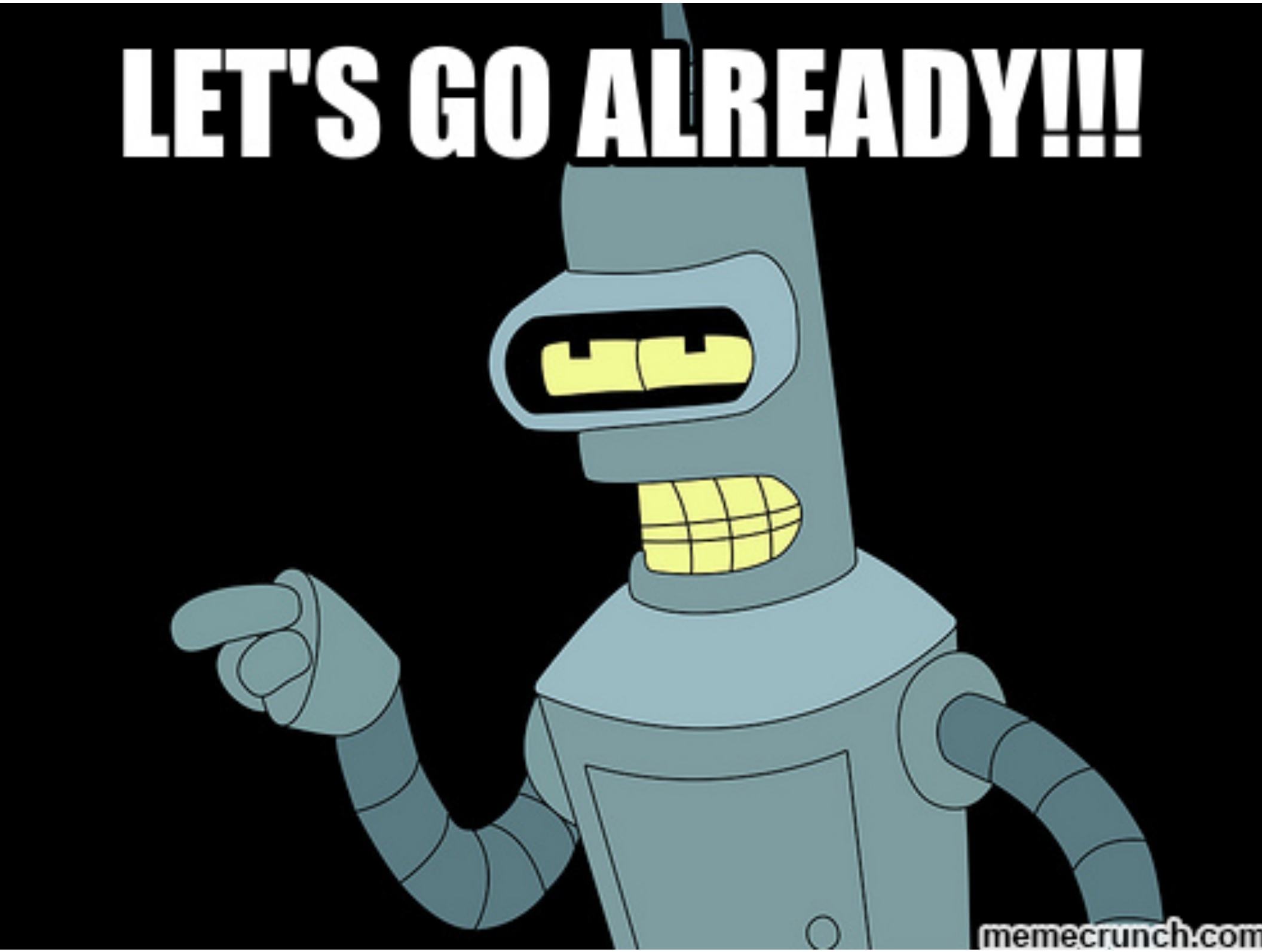
Testing and LLMs

- Given the nature of their outputs if deving with LLMs we must take extra care testing the results. We can't just use the code without inspection and verification
- Further if you build a system relying on LLM data how do you reasonably test around a system that inherently isn't deterministic?
- Consider that unit tests and most other automated tests rely on strict input X produces result Y where these systems are more like input X produce result Y* where * suggests variations

Testing and LLMs

- One possibility to deal with untrustworthiness of LLMs for code generation is to reverse the problem
- Following a TDD thought we would generate the tests (red) (effectively a spec really) and then have the LLM generate the code to pass the tests (green).
- Assuming the tests were well designed this would work fairly well, though do note the refactor step might be very important as generated code might literally be hardcoded just to pass!

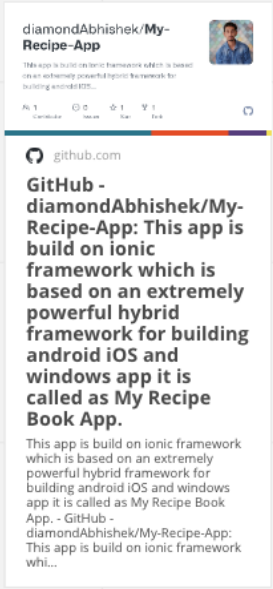

Don’t jump across an unknown GAP unless you know you can make it!
AI Risk
Reduce It!
Idea: Try Generative AI tech for text, code, image, video, etc. and see what it can or can’t do for us BEFORE we do the bigger project

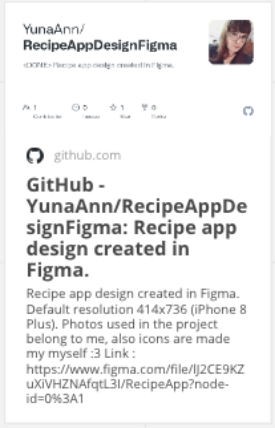

LET'S GO ALREADY!!!
memecrunch.com

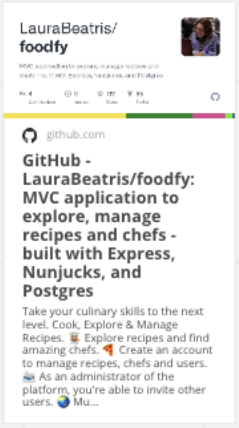
Let's Goooooooo!

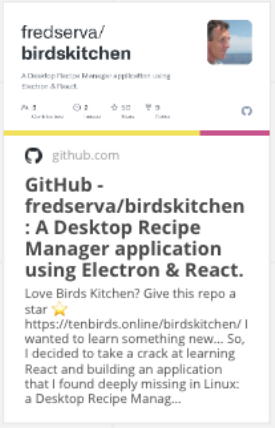
- Finding an implementation is easy enough even with source and we can do a ChatGPT session or do a Github search right now, app store search, etc. and get our calendar web component
- Bigger Spoiler: **Most things** have been done already in some fashion broadly but the specifics may not have been done
- Heed the mistakes of past quarters. The best teams studied everything online, but they did their OWN thing
  - Also this is not an AIP issue this is a thinking and influence worry that can plague your whole career

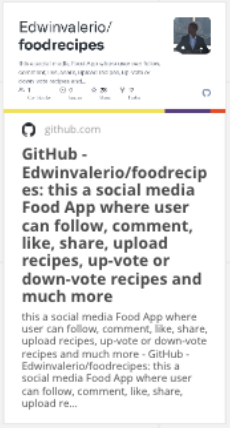
Let's Gooooooo! Danger

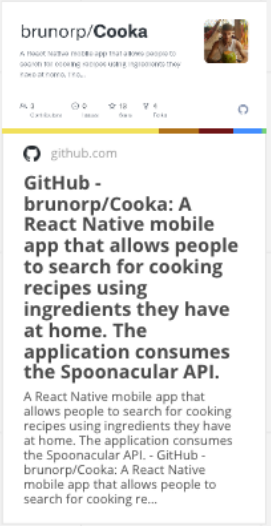


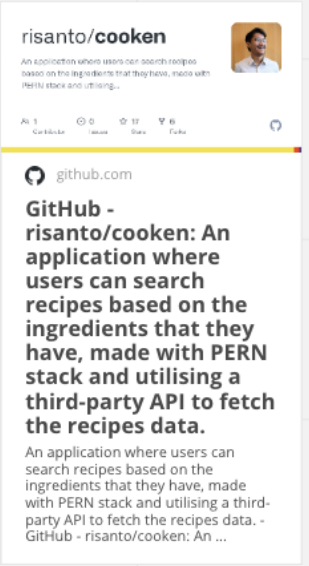




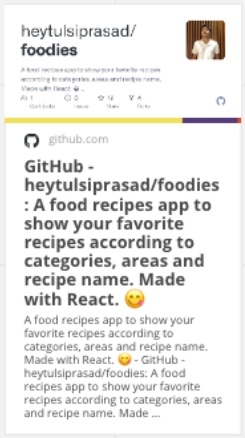


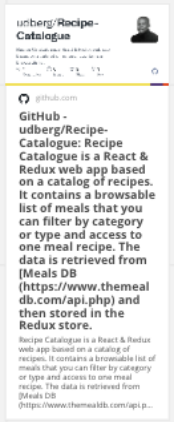


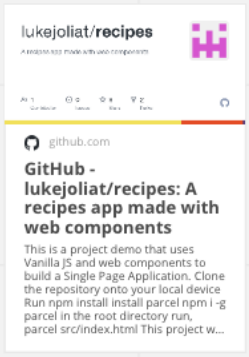




https://www.figma.com/file/lJ2CE9KZuXiVHZNAfqtL3I/RecipeApp?node-id=0%3A1







THIS IS COULD BE AN AIP TRAP HERE OR A GOLD MINE OF DECEPTION OR VALUE , ALSO SEE App Store, etc.

Topic Thoughts to Start

- Clearly there are different types of app ideas world wide and we have a pretty diverse class so there could many things. Begs localization!

- The steps of what we may capture influenced by Agile have a bit of ceremony and context. How I present things might be just as important as other aspects of the algorithm - SPOILER: Iceberg Theory

Think…then code

- Brain Storming, Requirements, and Specs before Code.
  - Code First Ask Questions Later = Explosion
  - Does this mean we can't code now?
  - Yes and no…enter the idea of exploratory coding and PoCs and other ideas, but be careful those MUST be thrown away. PoCs are not product and the initial decisions made with such things are often crazy limiting

…but sometimes code to think

- As just mentioned in some situations you may find that you can't imagine things without trying things in code. To address this be willing to do some exploratory programming hence the 8-ball apps!

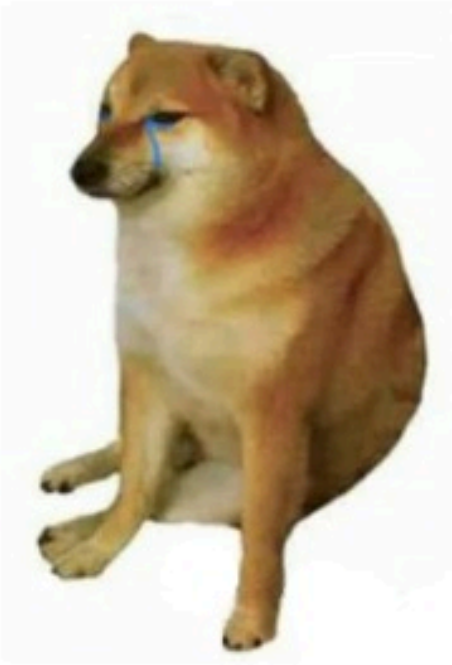
- Exploratory efforts might suggest you peel of member(s) to try something out and write some throw away code or use a dependency to prove if something is hard or not.
  - Extreme Caution: Be very careful not to attempt to morph a code exploration into a finished product! Be willing to throw things away, but retain the learning!

Careful…Danger Ahead

Follows path takes "tutorial level" seriously

Crushing the SE Course

Assumes ease and avoid advice



Gets crushed by SE Course

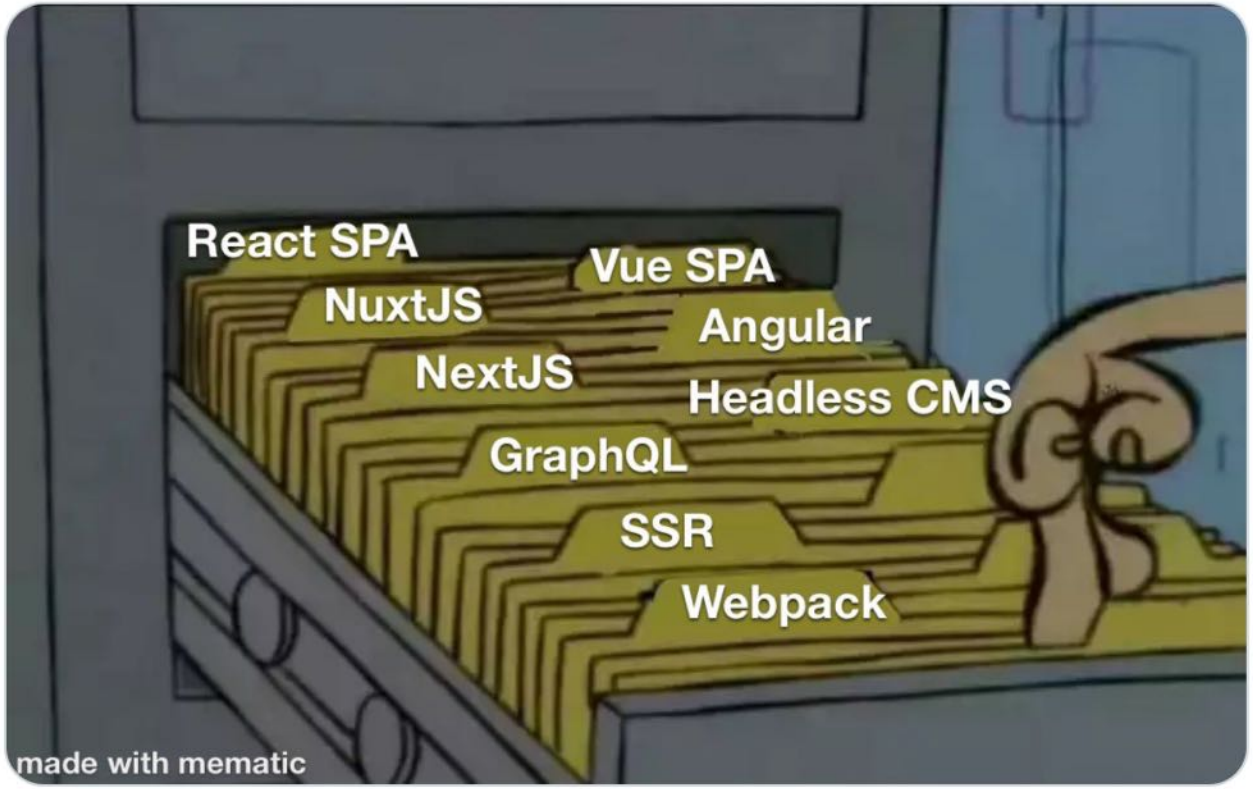
imgflip.com

Extra Complexity Considered Harmful*

Una Kravets Retweeted

Chris⚡Arter @ChrisArter · 15h

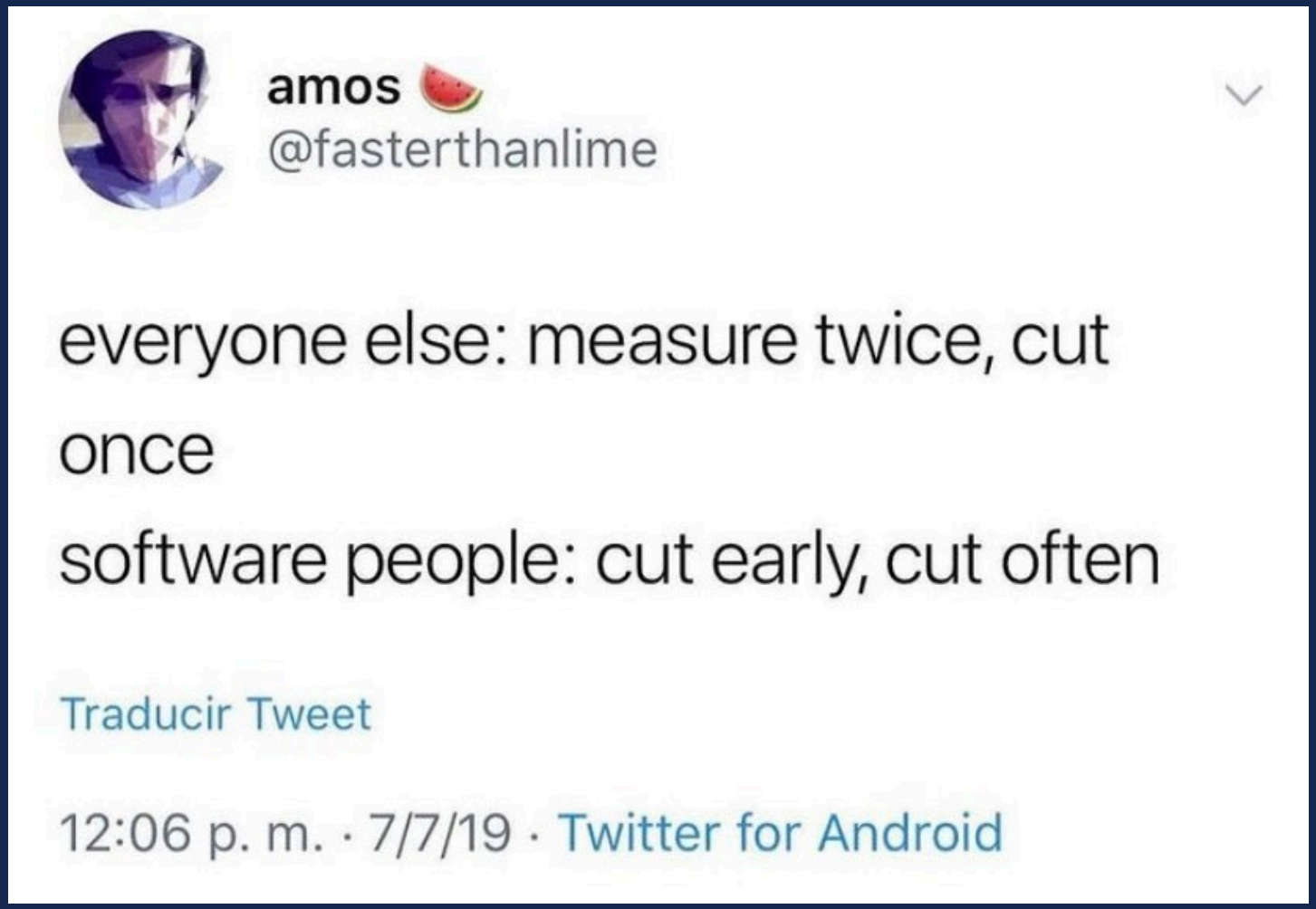
Me deciding how I'm going to over-engineer a Simple web app

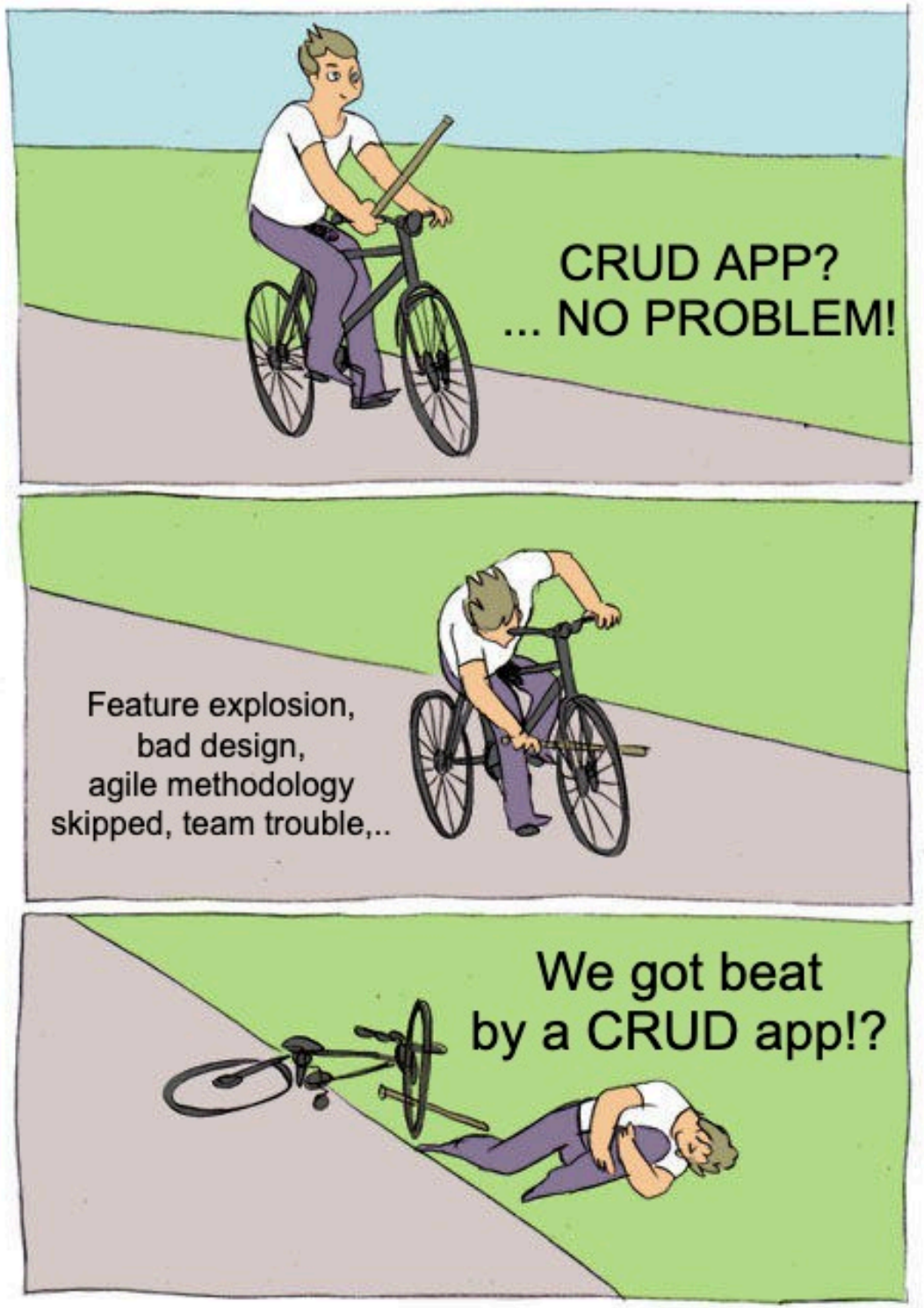


23 239 1.5K

* to your free time, stress, grade, etc.

Warning Not to Be Taken Lately





Project Reveal

Hours and Days Now Saves Grades, Stress and Work Weeks Later

Prompt

Raw Materials

Design Process
Brainstorming

Feature Thinking

User Thinking

Planning

Proofing and Prototyping

App Reasearch

Principles

Brainstorm

Roadmap

Code Demos

PoC Wireframes

Wild Digital Analo...

Resourcing

CSS, Font and JS Resource...

Time To Finalize
and Break True Ground!

2% Zoom !!!

2%

Project Reveal


MARIO
000000
×00
WORLD
1-1
TIME
LEVEL UP
IF YOU MASTER THIS PHASE YOU...
imgflip.com

You Then Realize Even This Simple Level Is More Like…

Project Reveal

//CSE
110;

Summary

- We should admit that our problem is pretty mundane and that is a complicated (by domain and humans) and hopefully not a complex or chaotic effort (though it can become that)
- Disrespecting the domain is the first chance for a critical flaw to be baked into our software
- Controlling the process, communicating, collaborative building, and always valuing humans over tech, tools and code is key to successful outcomes.
  - I do acknowledge ignoring this advice may be the only way for some to come to believe it and the class has baked (ha ha!) that in.

Coming Up in CSE 110

- Weekend and Next Week
  - Teams working on their intro activities & video
  - Teams starting in on the warm-up
  - In the Back of the Mind, thinking about the app (do not start on it yet, though!)

- Lecture / Labs Next Week
  - More HTML, CSS, Git
  - Introduction to Play Styles, Process Models, and Design and Ideation for eventually doing the app AFTER the warm-up